最小二乘方法：Python 交互学习

Least-Squares-Methods-in-Python · 中文初版 v0.1

Ancho5515 / Least-Squares-Methods-in-Python

2026 年 11 月 6 日

目录

阅读与运行指南 **1**

0.1 怎样读，才不会一上来就被公式拦住 1

0.2 先认识四个会反复出现的词 1

0.3 先把工具准备好 2

0.4 第一次运行：让电脑找回一条直线 2

0.5 怎样确认自己在理解，而不只是按运行键 3

0.6 安装和遇到问题时怎么办 4

0.7 先停一下，自己说一遍 4

第一章 从求平均到拟合直线 **5**

1.1 先看实验：最低的地方在哪里 5

1.2 想知道公式怎么来：平均数为什么赢 5

1.3 平方的脾气：特别在意大错误 6

1.4 同一个问题升级了：重量不变，小车却在移动 6

1.5 想知道公式怎么来：一次调两个数 7

1.6 图出来以后，先别急着说成功 8

1.7 动手试：一次只改一件事 8

第二章 如何建立最小二乘模型 **9**

2.1 先把四样东西分开 9

2.2 先预测，再运行 9

2.3 需要查公式时再看 10

2.4 线性不是指图上必须画直线 10

2.5 点很多，也可能回答不了你的问题 11

2.6 不同单位的误差，先换到可比较的尺度 11

2.7 一个看似省事的改写，可能换了比赛规则 11

2.8 动手试与检查 12

第三章 线性最小二乘的数学原理 **13**

3.1 为什么会出现“投影”这个词 . . . 13

3.2 想知道公式怎么来：最低点的条件 . . . 13

3.3 刚才的图，横纵轴到底是什么 . . . 14

3.4 为什么继续加参数，总能把训练分数压低 . . . 15

3.5 先猜，再试，再核对 . . . 16

第四章 如何可靠地求解线性问题 **17**

4.1 先认识条件数：小误差会被放大多少 . . . 17

4.2 想知道计算工具做了什么 . . . 17

4.3 别把某台电脑的最后几位当成定理 . . . 18

4.4 一个常见误读：空结果不等于没误差 . . . 19

4.5 动手试：把挤在一起的刻度拉开 . . . 19

第五章 噪声、权重与估计可信度 **20**

5.1 先用白话说权重 . . . 20

5.2 想知道公式怎么写 . . . 20

5.3 几次读数可能一起错，不能当成几份新证据 . . . 21

5.4 参数也会随读数波动 . . . 22

5.5 看懂散点云，不只看两个误差条 . . . 23

5.6 动手试与解答 . . . 23

第六章 正则化与过拟合 **24**

6.1 给“别为了追点而乱拐”一个明确规则 . . . 24

6.2 想知道公式怎么写 . . . 24

6.3 三批数据，做三份不同的工作 . . . 25

6.4 进阶解释：它为什么能压住不稳定的方向 . . . 26

6.5 动手试：给曲线松绑或收紧 . . . 26

第七章 进入非线性最小二乘 **27**

7.1 雅可比：一张“旋钮影响表” . . . 27

7.2 想知道公式怎么写：局部线性化 . . . 27

7.3 怎么读刚才的图 . . . 29

7.4 不会求导，也可以试着量；但别把步子缩到看不见 . . . 29

7.5 自己试与核对 . . . 29

第八章　迭代优化的基本工具 **31**

8.1 先找下坡，再试脚下 . . . 31

8.2 想知道判断标准：不能只是碰巧降了一点 . . . 31

8.3 牛顿法：不仅问往哪儿降，还估计地形怎样弯 . . . 32

8.4 程序停下了，可能有几种完全不同的意思 . . . 33

8.5 动手试与解答 . . . 33

第九章　Gauss–Newton 方法 **34**

9.1 先看它会犯什么错 . . . 34

9.2 想知道公式怎么来 . . . 34

9.3 给 GN 加一个“先试再走”的习惯 . . . 35

9.4 卡住时，先问地图和数据出了什么问题 . . . 36

9.5 自己试一下 . . . 37

第十章　Levenberg–Marquardt 方法 **38**

10.1 想知道公式怎么写：罚这一步，不是罚最终答案 . . . 38

10.2 地图说能省 10 分，真的省了多少 . . . 39

10.3 刹车有用，但不是自动驾驶的全部 . . . 40

10.4 动手试与解答 . . . 40

第十一章　信赖域与狗腿法 **41**

11.1 狗腿法：沿一条折线路线选步 . . . 41

11.2 想知道公式怎么来 . . . 41

11.3 一圈画多大，要听实际结果 . . . 42

11.4 不要从一张曲线宣布速度冠军 . . . 43

11.5 动手前先确认你在改哪个实验 . . . 44

第十二章　读数不完美时：单位、边界和异常点 **45**

12.1 先把旋钮的刻度说清楚 . . . 45

12.2 有些位置根本不允许走 . . . 45

12.3 不让几个离谱读数牵着整条线走 . . . 45

12.4 降低影响以后，仍然要调查异常点 . . . 46

12.5 结果不对时，按这个顺序排查 . . . 47

12.6 动手试：把正常波动与异常程度分开 . . . 47

第十三章　递推最小二乘 **48**

13.1 把一次更新拆成三句话 48
13.2 想知道公式怎么写 48
13.3 如果对象真的变了，就不能永远平均旧答案 49
13.4 它和卡尔曼滤波为什么像，又为什么不能随意画等号 50
13.5 公式写得相等，电脑算起来未必同样稳定 50
13.6 动手试与解答 51

第十四章　综合案例：曲线拟合与标定　52

14.1 拟合前，先写清实验约定 52
14.2 起点从数据来，不偷看模拟答案 52
14.3 曲线好看，三个参数就都测准了吗 53
14.4 图上的带子，到底包住什么 53
14.5 把结果写成别人能复查的话 55
14.6 自己试与核对 55

第十五章　综合案例：PWM→XYZ 校正　56

15.1 先翻译名字，再谈算法 56
15.2 本实验故意简化了哪些地方 56
15.3 先轻轻试旋钮，量出这附近的响应 57
15.4 把前面几章连起来 58
15.5 没达到目标时，先说清为什么停 59
15.6 达标以后，也要重新验一次 60
15.7 动手试与解答 60

附录 A　必要数学基础　61

A.1 一排数、一张表：向量与矩阵 61
A.2 点积和范数：比较方向，汇总大小 61
A.3 导数、梯度、雅可比、Hessian：四种变化记录 61
A.4 正定、特征值、奇异值：碗是否朝上，哪些方向看得清 62
A.5 平均位置与共同波动：期望、方差、协方差 62
A.6 拿正文里的式子检查一次 63

附录 B　符号、公式与算法选择　64

B.1 先按遇到的困难找工具 64
B.2 符号速查：先说人话，再看维度 64

B.3 六句能让结果被复查的话 . . . 65
B.4 公式速查，第二遍阅读时再用 . . . 65
B.5 下面这段诊断，先看哪几项 . . . 65
B.6 检查题 . . . 66

附录 C 进阶方法的入口 **67**
C.1 能直接算的先算：可分最小二乘 . . . 67
C.2 内层答案也会跟着变 . . . 68
C.3 用走过的一步更新地图：割线条件 . . . 68
C.4 为什么还会有混合与拟牛顿方法 . . . 69
C.5 检查题 . . . 69

附录 D 参考来源、对应关系与勘误 **70**
D.1 主要来源 . . . 70
D.2 只想学会使用，先读到哪里 . . . 70
D.3 与 2004 年原书的对应 . . . 70
D.4 如何处理原书已记录的疑点 . . . 71
D.5 验证记录和适用范围 . . . 71
D.6 检查题 . . . 72

阅读与运行指南

你把同一个苹果放到秤上，先后看到 150 克、152 克、149 克。苹果没有变，读数却在变。你会用哪个数代表它的重量？如果要根据几次位置读数猜一辆小车的速度，又该怎么办？

这本书从这些问题出发。**最小二乘，就是先规定怎样给误差打分，再找出总分最低的那组设置**。“二乘”是平方的旧说法，例如 $3^2 = 3 \times 3 = 9$。它不是一种能自动找出真相的魔法，而是一套把猜测、测量和检查连接起来的办法。

不需要先学完大学数学。你可以先会加减乘除、读简单折线图，边运行例子边补知识。不过，运行代码仍需要一点 Python 基础：变量是给数起名字，数组是一排数，函数是可反复使用的计算步骤。如果这些也陌生，先看 零起点阅读指南，再回到这里。学历不是门槛；需要补的是具体知识。

0.1 怎样读，才不会一上来就被公式拦住

第一次阅读，先读每章的生活问题、操作提示和读图说明。标着“想知道公式怎么来”的部分可以暂时跳过；它们保留了推导，方便以后查。**会解释现象，比会背算法名字更早一步，也更重要**。

建议先读第 01、02 章：会算平均、会拟合直线、会说清误差是什么。再读第 05、06 章：比较可靠与不可靠的测量，看看曲线为什么会“记住错题”。第 03、04 章补上矩阵和可靠计算；第 07—11 章解释电脑怎样一步步调整。第 12—15 章把前面的办法连成完整任务。附录 A和白话词典随用随查，附录 C留到进阶时再读。

本书参考 Roger Labbe 的 *Kalman and Bayesian Filters in Python* 的教学思路：从读者能想象的问题出发，让文字、实验和结果互相解释。这里的故事和讲解按最小二乘问题原创组织，并非对其正文的翻译。非线性理论的主要参考资料见附录 D。

0.2 先认识四个会反复出现的词

- **观测**：实际读到的数，例如秤显示的 152 克。
- **模型**：我们暂时采用的解释，例如“苹果的重量一直没变”。
- **参数**：模型中还要决定的数，例如猜测的苹果重量。
- **残差**：模型算出的数减去实际读数。猜 151、读到 152，残差就是 -1；负号表示猜低了。

把所有残差各自平方，再相加，就是本书的基本评分。为了后面求导方便，再乘 1/2；所有候

选的分数一起减半，冠军不会变。写成公式是

$$r_i(x) = f_i(x) - y_i, \qquad F(x) = \frac{1}{2}\sum_{i=1}^{m} r_i(x)^2 = \frac{1}{2}\|r(x)\|_2^2.$$

这里 x 是待调的设置，y_i 是第 i 次读数，$f_i(x)$ 是模型对这次读数的预测，m 是读数个数。$\sum$ 表示 “逐个相加”；$\|r\|_2^2$ 只是 “这一排误差的平方和” 的简写，先不必背它。

本书始终用 “模型减观测”。换成 “观测减模型” 也能得到相同平方分数，但计算变化率的代码也要一起换符号，不能只改一半。

0.3 先把工具准备好

下面是公共准备代码，每章都有一份。它负责找到本书目录、载入计算与画图工具、设置图的大小。第一次可以先运行，不用逐行弄懂。运行后没有新图或打印文字，通常是正常的。

所有例子使用本地数据或程序生成的数据；依赖安装完成后，章节计算无需联网。

```
from pathlib import Path
import sys
root = Path.cwd().resolve()
while not (root / 'lsq').is_dir():
    if root == root.parent:
        raise RuntimeError('Run inside the repository')
    root = root.parent
if str(root) not in sys.path:
    sys.path.insert(0, str(root))
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import least_squares
from lsq import (finite_difference, whiten, gauss_newton,
                 levenberg_marquardt, dogleg, recursive_update)
np.set_printoptions(precision=5, suppress=True)
plt.rcParams.update({'figure.figsize': (6.4, 3.3), 'font.size': 10,
                     'axes.grid': True, 'savefig.dpi': 130})
```

0.4 第一次运行：让电脑找回一条直线

下面先造出五个点：时间是 0、1、2、3、4，对应数值是 1、3、5、7、9。规律是 “从 1 开始，每过一个时间单位增加 2”。我们假装不知道这两个数，让电脑从点中找回来。

在 Jupyter 中，点选代码框，按 **Shift+Enter** 运行并进入下一格。正常应看到 `intercept, slope : [1. 2.]`：`intercept` 是起始值，`slope` 是每单位时间增加多少。图中的圆点叫 `data`，表示数据；直线叫 `fit`，表示拟合结果。两者重合，因为这次故意没有加入测量误差。

先猜一猜：若每次增加 3，会找出什么？然后把 `y = 1 + 2*t` 改成 `y = 1 + 3*t`，**并把紧随其后的断言预期 `[1, 2]` 改成 `[1, 3]`**，再运行。`assert` 是作者放的自检，不是算法；只改造数规则而不改预期，

出现 `AssertionError` 正是在提醒两者不一致。试完可以恢复原值。

```
t = np.arange(5., dtype=float)
A = np.column_stack([np.ones_like(t), t])
y = 1 + 2*t
x, _, rank, singular = np.linalg.lstsq(A, y, rcond=None)
print('intercept, slope:', x, 'rank:', rank)
assert np.allclose(x, [1, 2])
plt.plot(t, y, 'o', label='data')
plt.plot(t, A@x, label='fit')
plt.xlabel('t'); plt.ylabel('y'); plt.legend(); plt.show()
```

```
intercept, slope: [1. 2.] rank: 2
```

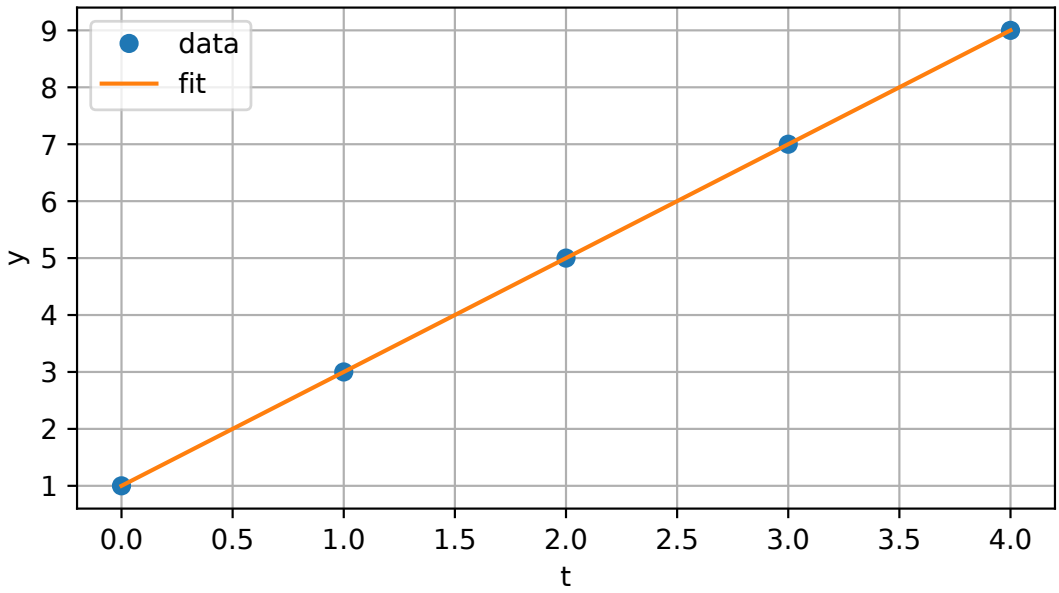


图 1: 环境验证：直线观测与拟合

0.5 怎样确认自己在理解，而不只是按运行键

你能说出“我在调哪几个数、程序按什么给误差打分、图上的点和线各是什么”，就已经完成了第一步。以后再逐渐补上：单位是否一致、剩下的误差有没有规律、数据是否足以分清这些参数、程序为什么停下。

例如，把上面的时间全部改为零，相当于只在起跑瞬间拍五张照片。照片再多，也看不出车跑得多快。数学上叫“秩亏”：数据没有提供足够的独立信息。程序可能仍返回一个斜率，但这个数不是被数据测出来的。想动手时先注释掉原来的断言，因为它只适用于原实验。

0.6 安装和遇到问题时怎么办

需要 Python 3.11 或更新版本，推荐 3.12。按仓库 README 的步骤建虚拟环境，安装 `requirements.txt` 和 JupyterLab，然后在项目目录运行 `python -m jupyterlab`，打开 `notebooks/00-guide.ipynb`。不要只运行中间一格：出现 `NameError` 时先从本章第一格重新运行；出现 `ModuleNotFoundError` 时先核对所选 Python 环境和依赖安装。

开发时用 `python -m unittest discover -s tests -v` 检查现有代码。`python tools/build.py --execute --test` 会执行全部章节并重写 Notebook 的保存输出；自己的实验副本更适合反复修改。中文 PDF 构建另需 Pandoc 和带中文组件的 TeX Live，完整命令见 README。这是制作书籍的工具，不是读懂第一章的前提。

0.7 先停一下，自己说一遍

问题：为什么不能先把正负误差加起来，再比较总数？

提示：一个猜高 2，一个猜低 2。

解答：直接相加会得到零，好像完全没错；各自平方后是 $4+4=8$，两次偏差都被记住了。平方只是本书先采用的一种评分规则，第 12 章会看到它也有不足。

第一章　从求平均到拟合直线

你用厨房秤称同一份豆子，五次读数是 10.1、9.9、10.2、10.0、9.8 克。你想在记录本上只写一个数。写 10.2，是相信最高的一次；写 9.8，是相信最低的一次。没有证据说明哪一次更可靠时，我们通常取平均，得到 10.0。

但“大家都这么做”还不是解释。我们先约定一个比赛规则：**每个候选重量，都要为它与五次读数的差距付分；差距先平方，再相加。分越低越好。**

只看 9 和 11 两个读数，就能手算。猜 9 时，分数为 $0^2+(-2)^2=4$；猜 10 时是 $1^2+(-1)^2=2$；猜 11 时又是 4。站在中间，分摊了两边的误差。这说明的是“在这种评分下最合适”，没有证明真重量一定是 10。如果秤每次都多报 1 克，取再多次平均也不会自动修好它。

1.1　先看实验：最低的地方在哪里

下面把 9.6 到 10.4 之间的许多候选重量逐个打分。横轴 `candidate mass` 是猜的重量；纵轴 `half squared error` 是平方误差和的一半。找曲线最低点，看看它是否落在平均值那条竖线上。程序应打印 `mean: 10.0`。

1.2　想知道公式怎么来：平均数为什么赢

把候选重量记为 x，第 i 次读数记为 y_i，读了 m 次。分数写成 $F(x)=\frac{1}{2}\sum_i (x-y_i)^2$。它的导数

$$F'(x)=\sum_i (x-y_i)=mx-\sum_i y_i$$

是在问：“把猜测稍微往上调，分数是增是减？”令它为零，得到 $x^*=\bar{y}$，即平均数。$F''(x)=m>0$ 表示这条评分曲线像碗，碗底是唯一最低点。

没学过导数也有另一条路：

$$\sum_i (x-y_i)^2=\sum_i (\bar{y}-y_i)^2+m(x-\bar{y})^2.$$

左边是任意猜测的分数；右边是平均数的分数，再加一笔“离开平均数”的费用。后面那项不会小于零，因此换成其他数只会持平或变差。

```
y = np.array([10.1, 9.9, 10.2, 10.0, 9.8])
grid = np.linspace(9.6, 10.4, 201)
loss = .5*np.sum((grid[:, None]-y)**2, axis=1)
print('mean:', y.mean(), 'minimum cost:', .5*np.sum((y-y.mean())**2))
assert np.isclose(y.mean(), 10.)
plt.plot(grid, loss); plt.axvline(y.mean(), color='C1', label='mean')
plt.xlabel('candidate mass'); plt.ylabel('half squared error'); plt.legend(); plt.show()
```

```
mean: 10.0 minimum cost: 0.04999999999999964
```

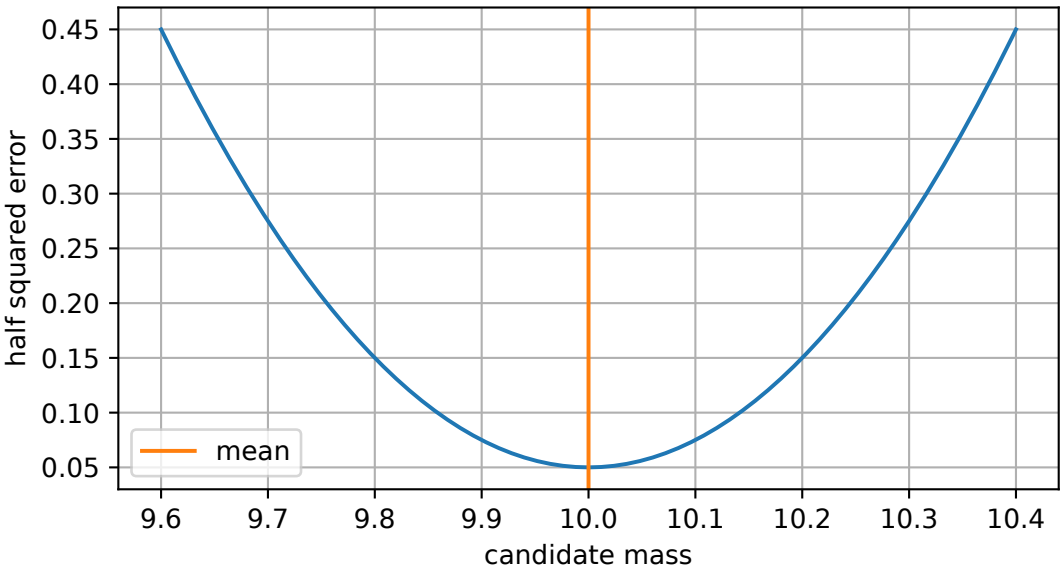


图 1.1: 常数估计的平方损失在平均值处最小

1.3 平方的脾气：特别在意大错误

先猜：五个读数里有一个被错录成 20，平均数会不会被拉过去？会，因为偏差从 1 增到 10，平方罚分从 1 增到 100。这个规则不区分“大偏差来自重要信息”还是“来自一次录错”。

如果改成把每次差距的绝对值相加，也就是只去掉正负号，最合适的数会变成**中位数**：把读数排好队，取中间的那个。哪种规则更合适，要看数据出了什么问题，第 12 章会继续比较。

1.4 同一个问题升级了：重量不变，小车却在移动

现在每隔一会儿给小车拍照。我们关心两个数：开始时在哪儿，以及每个时间单位前进多少。暂时用匀速来解释：

$$\text{位置} = \text{起点} + \text{速度} \times \text{时间}, \qquad f_i(a, b) = a + bt_i.$$

a 叫**截距**，是时间为零时的预测位置；b 叫**斜率**，是每单位时间的增量。它们就是要调的两个

参数。找一条线，让每个点到线的竖直差距的平方和最小，叫**拟合直线**。线不必穿过每个点，因为拍照读数有误差。

下面代码先造出起点 1.2、速度 2.0 的小车，再给读数加一点随机波动。`hand formula` 是按下方公式算的，`NumPy` 是成熟工具算的，两者应接近。左图是点与直线，右图是每次“预测减观测”的差。

1.5 想知道公式怎么来：一次调两个数

让分数分别对 a、b 的变化率为零，得到

$$\sum_i (a + bt_i - y_i) = 0, \qquad \sum_i t_i(a + bt_i - y_i) = 0.$$

只要拍照时间不全相同，就有

$$b = \frac{\sum_i (t_i - \bar{t})(y_i - \bar{y})}{\sum_i (t_i - \bar{t})^2}, \qquad a = \bar{y} - b\bar{t}.$$

横线上面的符号表示平均值。分子记录“时间高于平均时，位置是否也高于平均”；分母记录“时间散得有多开”。如果所有时间一样，分母为零，就没法从静止的一瞬间推断速度。

这里量的是**竖直方向的位置差**，不是点到线的最短距离；我们暂时把时间当作足够准确的输入。时间也有明显误差时，需要重新选择模型。

```
rng = np.random.default_rng(101)
t = np.linspace(0, 5, 25)
y = 1.2 + 2.0*t + rng.normal(0, .5, len(t))
b = np.sum((t-t.mean())*(y-y.mean())) / np.sum((t-t.mean())**2)
a = y.mean()-b*t.mean()
A = np.column_stack([np.ones_like(t), t])
reference = np.linalg.lstsq(A, y, rcond=None)[0]
print('hand formula:', [a, b], 'NumPy:', reference)
assert np.allclose([a, b], reference)
fig, ax = plt.subplots(1, 2, figsize=(7, 3))
ax[0].scatter(t, y); ax[0].plot(t, a+b*t, color='C1')
ax[0].set(xlabel='time', ylabel='position')
ax[1].stem(t, a+b*t-y); ax[1].set(xlabel='time', ylabel='residual')
plt.tight_layout(); plt.show()
```

```
hand formula: [np.float64(1.2468646986979914), np.float64(1.9670997346181176)] NumPy: [1.24686 1.9671 ]
```

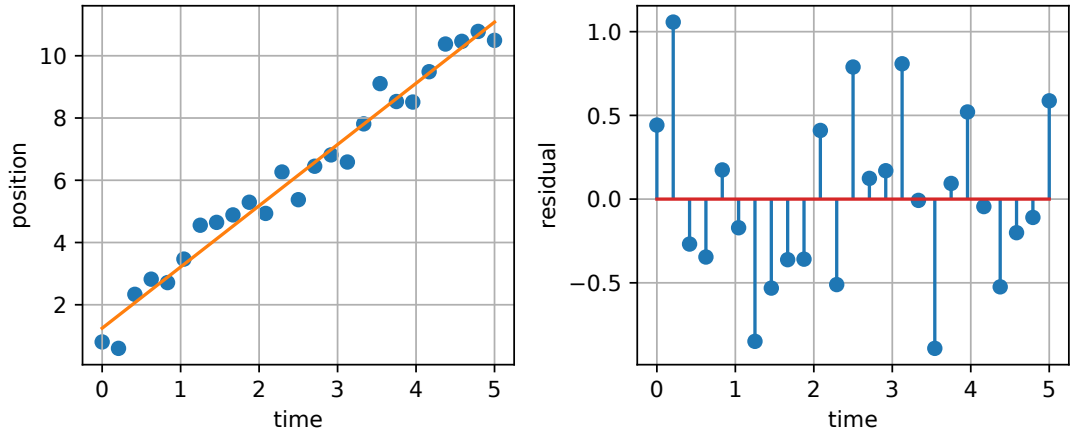


图 1.2: 直线拟合及其残差

1.6 图出来以后，先别急着说成功

右图的 `residual` 就是残差：在线上方的点，模型比观测低，所以残差为负；在线下方则为正。理想情况下，它们围着零上下散开。如果出现一段连续弯曲，可能是小车在加速，直线不够用了；后半段起伏特别大，可能是后面的测量更不可靠。

对包含截距的普通最小二乘直线，所有残差相加应接近零。但**加起来是零，不表示每次都没错**，也不保证直线是正确模型。

1.7 动手试：一次只改一件事

1. 把造数据那行的起点 `1.2` 改成 `4.2`。先猜：直线会整体抬高，还是变陡？运行后比较 `a` 与 `b`。
2. 恢复起点，再把噪声大小 `.5` 改成 `1.5`。先猜：散点离直线会更远吗？残差图会怎样？
3. 把某个 `y` 增加 8，分别试时间靠中间和靠两端的点。每次重新运行整格，以免把修改累加到上一次数据上。

提示：第三个实验可以在造好 `y` 后、计算 `b` 前加 `y[0] += 8`，再改为 `y[len(y)//2] += 8` 比较。

解答：第一项只让截距增加 3，斜率不变；第二项使读数波动变大，但单次结果如何变化要看随机样本；第三项中，两端点通常更容易把线“撬斜”，这叫**杠杆作用**，好比在尺子的远端施力更容易改变方向。

再想一想：把秒换成毫秒，斜率数值会变成原来的 1/1000，但小车没有变慢，只是计时单位变了。能区分“数变了”和“物理现象变了”，比记住公式更有用。

第二章　如何建立最小二乘模型

一杯热饮正在变凉。刚放下时降温快，过一会儿降温慢，最后接近室温。你想知道“再等一分钟，大概会凉到什么程度”。在打开求解器之前，先要告诉它：你准备拿什么规律解释这些读数？

建模就是选一个暂时的解释，并说清它怎样接受检验。同一批点可以画直线，也可以画曲线；电脑只会替你调好所选曲线，不会自动判断这个解释有没有道理。

2.1　先把四样东西分开

拿“逐渐下降并趋于平稳的响应”作例子：

我们知道或要找的东西	本章名字	代码中的位置
在什么时候测量	输入	`t`
每次读到了多少	观测	`y`
起初高出背景多少、下降多快、最终接近多少	参数	`a, k, c`
按参数算出的响应	模型预测	`model(p)`

下面用 $f(t) = a\exp(-kt) + c$ 表示这种下降。`exp` 是指数函数；先把它理解成一个“开始下降得快、后来渐渐变缓”的零件。a 管起初高出背景的部分，k 管下降速度，c 管最终靠近的水平。**这只是同类现象的一种简化模型，不能因此断言任意热饮或传感器都遵守它**。

每个读数都有一个差值：`model(p) - y`。把这些差值排成一列，叫**残差向量**。本章有 35 个读数、3 个参数，所以你应看到残差的 `shape` 是 `(35,)`，参数是 `(3,)`。`shape` 只是数组大小，不是什么新的数学运算。

2.2　先预测，再运行

只调大 c，整条曲线会抬高；只调大正的 k，曲线会更快靠近背景。下面程序生成一组已知答案的模拟数据，再假装不知道答案去拟合。`truth` 是造数据时用的设置，`estimate` 是找回的设置。它们应接近，却不必完全相等，因为读数加入了噪声。

2.3 需要查公式时再看

一般写法是 $y_i = f(t_i, x) + \varepsilon_i$：观测等于模型预测，再加测量误差 ε_i。$x \in \mathbb{R}^n$ 只是说 “有 n 个可以取实数的待调数”。

$$r_i(x) = f(t_i, x) - y_i, \qquad x^* = \arg\min_x \frac{1}{2}\|r(x)\|^2.$$

`arg min` 读成 “找出使后面分数最小的那组参数”。它返回的是参数，不是最低分本身。

2.4 线性不是指图上必须画直线

$a + bt + ct^2$ 虽然可以画出弯曲的抛物线，但未知数 a, b, c 都只是乘在已知零件前面，因此叫**对参数线性**。而 $a\exp(-kt) + c$ 的 k 藏在指数里面，叫非线性。前者通常能一次求解，后者往往要从一个猜测出发，反复调整。如果 k 已知，只找 a, c，又可以一次线性求解。

```
t = np.linspace(0, 4, 35)
rng = np.random.default_rng(102)
truth = np.array([2.5, .8, .4])
def model(p):
    a, k, c = p
    return a*np.exp(-k*t)+c
y = model(truth)+rng.normal(0, .04, len(t))
def residual(p):
    return model(p)-y
fit = least_squares(residual, [2., .5, .2], bounds=([0, 0, -np.inf], np.inf))
print('truth:', truth, 'estimate:', fit.x)
print('residual shape:', residual(fit.x).shape, 'parameter shape:', fit.x.shape)
plt.scatter(t, y, label='measurements')
plt.plot(t, model(fit.x), color='C1', label='exponential fit')
plt.xlabel('time'); plt.ylabel('response'); plt.legend(); plt.show()
```

```
truth: [2.5 0.8 0.4] estimate: [2.51287 0.82961 0.42906]
residual shape: (35,) parameter shape: (3,)
```

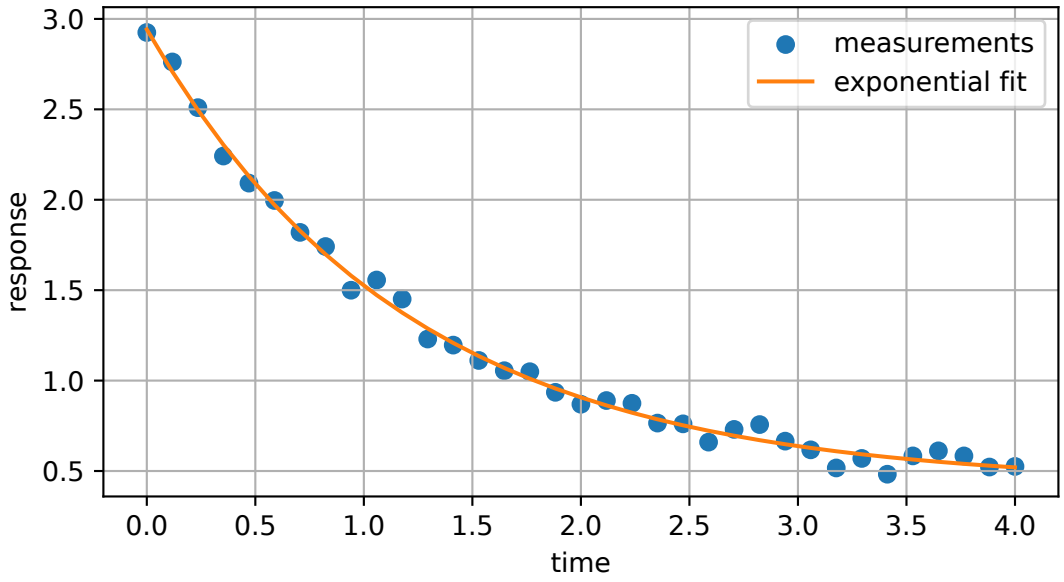


图 2.1: 指数模型拟合合成衰减观测

2.5 点很多，也可能回答不了你的问题

假如只在 $t = 0$ 测量，再重复一千次，模型都只能给出 $a + c$。我们知道两袋豆子合起来多重，却不知道每袋各多重；k 更是根本没起作用。这叫**参数不可辨识**：现有数据分不清不同设置。增加不同时间、不同条件的观测，往往比继续重复同一种观测更有帮助。

2.6 不同单位的误差，先换到可比较的尺度

一组误差用毫米表示，另一组用伏特表示，不能直接用数值大小判断谁更严重。同样是差 1，对通常波动 0.1 的仪器很大，对通常波动 10 的仪器却很小。

常用办法是“误差除以通常的波动大小”：$\tilde{r}_i = r_i/\sigma_i$。$\sigma_i$ 叫**噪声标准差**，大致描述同样条件下读数常见的波动尺度。处理后，差 2 就可以理解成“偏了两个波动尺度”。如果几次读数会一起偏高或偏低，还要一起处理，第 05 章会讲“白化”。

2.7 一个看似省事的改写，可能换了比赛规则

对指数模型取对数，有时能把曲线写得像直线。但对 $y - c$ 取对数后，原来相等的误差会变得不一样大；$y \le c$ 时还无法取实数对数。**改写模型表达式和保留同一种误差评分，是两件不同的事。**不能只因为新公式好算，就当它回答了原来的问题。

2.8　动手试与检查

先只改 `truth` 中的一个数，观察造出的曲线怎样变，再从同一个初值重新拟合。不要同时改三个数，否则难以知道变化从哪里来。

问题一：$a\sin(t)+b\cos(t)$、$\sin(at+b)$、$a/(1+bt)$，哪些对待估参数是线性的？

提示：先把已知的 t 当作已经算好的数，再看未知数有没有进入正弦、分母等零件内部。

解答：只有第一个。它只是把两种已知波形分别乘 a,b 后相加。

问题二：把残差改成 `y - model(p)`，为什么之后自己写导数时也要改符号？

解答：残差的正负含义反了，它随参数的变化方向也反了。只改一边，会把下降的路标装反。

离开本章前，试着说清自己的任务：输入是什么、测到什么、想调哪几个数、用什么给误差打分。拟合后还要拿没参与调参的数据检查预测。训练分数很低，可能只是参数太多，并不代表到新地方也猜得准。

第三章　线性最小二乘的数学原理

第 01 章只调起点和速度。假如以后要调三个、十个数，还要每次重写一套求解公式吗？我们需要一种把“每个参数怎样影响每次读数”整齐记下来的办法，这就是矩阵在这里的用途。

矩阵先当作一张计算表，不必先想成抽象对象。直线 $a+bt$ 在 $t=0,1,2$ 的预测，可以写成

$$\begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} a \\ a+b \\ a+2b \end{bmatrix}.$$

左边的表叫 A，待调的两数叫 x。**每行对应一次观测，每列对应一个参数的作用**。Ax 是让计算表按这些参数算出所有预测；代码里的 `A @ x` 做的就是这件事。

3.1　为什么会出现“投影”这个词

先把问题缩到只有两次观测、一个参数。假如模型只能给出 $(x, 2x)$，那么可选预测是 $(0,0)$、$(1,2)$、$(2,4)$……都在同一条线上。实际读数却是 $(2, 0.5)$，没有哪个 x 能同时完全满足两次读数。

我们要在线上挑一个离实际读数最近的点。把读数向这条线“垂直落下去”，落脚点就是最小二乘预测。这叫**正交投影**：正交就是垂直，投影就是在允许的位置里找最近点。这里距离的平方，恰好是两次残差的平方和。

下面图中，蓝箭头是真实读数，橙箭头是选出的预测，灰线是模型能到的位置，虚线连接两者。先看虚线是否垂直灰线，再看打印的 `orthogonality` 是否接近零。它是在用数字检查垂直关系。

3.2　想知道公式怎么来：最低点的条件

对一般计算表 $A \in \mathbb{R}^{m\times n}$，即 m 行、n 列，评分是

$$F(x) = \frac{1}{2}(Ax-y)^T(Ax-y), \qquad \nabla F(x) = A^T(Ax-y).$$

T 表示转置，也就是把表的行列交换；∇F 叫梯度，是“各个参数稍调一点，分数分别怎样变”的清单。最低点满足

$$A^TAx = A^Ty.$$

这叫**正规方程**，“正规”只是历史名称，不是说别的方法不正规。如果每列都提供其他列不能替代的信息，称为“满列秩”，参数解唯一；如果有重复或可替代的列，称为“秩亏”，不同参数可能给出同样预测。

实际计算先用 `lstsq`、QR 或 SVD。不要因为纸上能写逆矩阵，就在程序中直接算 $(A^TA)^{-1}$；第 04 章会看到这样做可能损失精度。

```
v = np.array([1., 2.]); y = np.array([2., .5])
x = (v@y)/(v@v)
projected = x*v
print('coefficient:', x, 'orthogonality:', v@(projected-y))
assert np.isclose(v@(projected-y), 0.)
plt.plot([0, 1.3], [0, 2.6], color='gray', label='column space')
plt.arrow(0, 0, *y, width=.02, color='C0', length_includes_head=True)
plt.arrow(0, 0, *projected, width=.02, color='C1', length_includes_head=True)
plt.plot([y[0], projected[0]], [y[1], projected[1]], '--', color='C2')
plt.axis('equal'); plt.xlabel('observation 1'); plt.ylabel('observation 2')
plt.legend(); plt.show()
```

```
coefficient: 0.6 orthogonality: 0.0
```

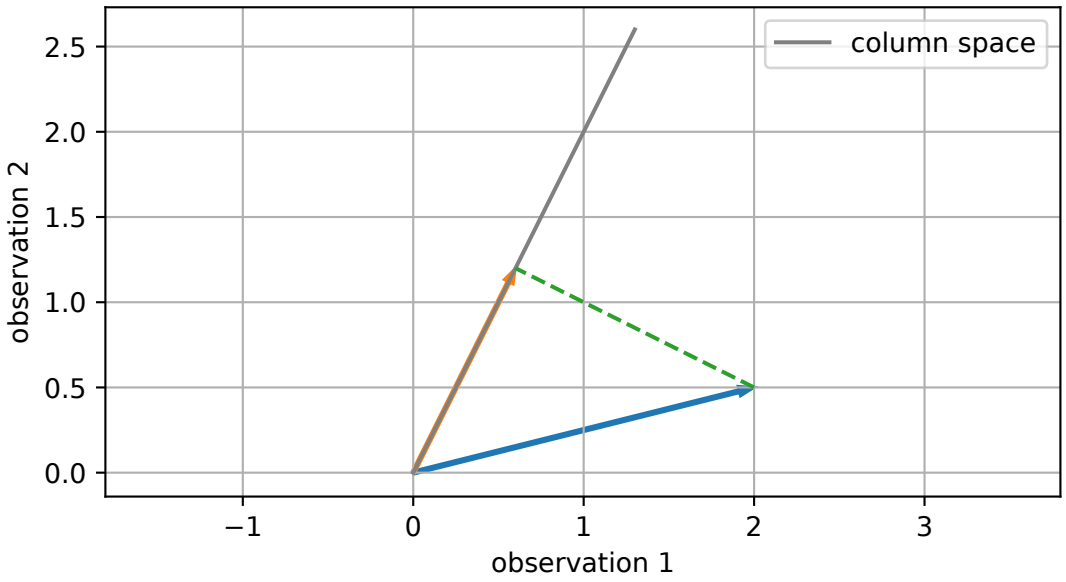


图 3.1: 观测向量在一维列空间上的正交投影

3.3 刚才的图，横纵轴到底是什么

两根轴不是两个参数，而是**第一次、第二次观测的预测值**。我们只有一个旋钮 x，却要照顾两次读数，所以模型能到的地方只是一条线。把所有允许预测的位置合起来，数学上叫 A 的**列空间**。你可以把它理解成“现有模型的活动范围”。

再看第 01 章的直线拟合。每个参数都带来一种可调的变化：截距把所有预测一起抬高；斜率

让它们按时间不同程度地增加。最优残差与这两种变化都垂直，写成 $A^T r = 0$。意思是：已经没有一个很小的参数调整能继续降低平方分数。

下面回到六次观测的直线拟合。先看 `A.T @ r` 的两个数是否都接近零，不必先弄懂 `qr` 内部怎样工作。`H @ y` 是把实际读数换成允许范围内最近预测的操作；公式为

$$H = A(A^T A)^{-1} A^T, \qquad H^T = H, \quad H^2 = H.$$

这段公式只适用于满列秩。$H^2 = H$ 说的是：已经落在线上的点，再投一次仍在原地。代码用 `Q @ Q.T` 实现同一个操作，避免显式求逆。

```
t = np.arange(6., dtype=float)
A = np.column_stack([np.ones_like(t), t])
y = np.array([1., 2.8, 5.2, 7.1, 8.9, 11.2])
Q, R = np.linalg.qr(A, mode='reduced')
x = np.linalg.solve(R, Q.T@y)
H = Q@Q.T
r = A@x-y
print('parameters:', x, 'A.T @ r:', A.T@r)
print('training residual norm:', np.linalg.norm(r))
assert np.allclose(A.T@r, 0., atol=1e-12)
assert np.allclose(H@H, H)
assert np.allclose(H@y, A@x)
```

```
parameters: [0.94762 2.03429] A.T @ r: [0. 0.]
training residual norm: 0.33580039422535635
```

3.4 为什么继续加参数，总能把训练分数压低

把原来的模型保留，再给它一个新旋钮，至少还可以把新旋钮设为零、沿用旧预测。因此，**在不另加惩罚的普通最小二乘中，增加模型选择不会让最低训练误差变大**。但这不保证新数据上更好；第 06 章会看它怎样过拟合。

想看严格理由时，用这个误差分解：

$$\|Ax - y\|^2 = \|Ax^* - y\|^2 + \|A(x - x^*)\|^2.$$

x^* 是已找到的最优参数。右边第一项是最低分，第二项是换参数后额外付出的平方距离，不能小于零。它来自两个互相垂直的方向，和直角三角形的勾股定理是同一种关系。

如果存在非零 z 却有 $Az = 0$，沿这个方向换参数不会改变任何预测。于是 x^* 和 $x^* + z$ 一样好。**预测能确定，不等于每个参数都能分别确定**。

3.5 先猜，再试，再核对

问题一：把 `A` 的某一列复制一遍，能得到更好的拟合吗？

提示：原来能调出的预测，现在是否出现了新种类？

解答：没有。只是两个旋钮做了同一件事。训练误差不变，参数却不再唯一。

问题二：为什么直线拟合的残差相加接近零？如果去掉截距呢？

解答：截距对应一列全 1，$A^T r = 0$ 的这一项就是残差之和。去掉截距后不再有这个保证。

能把本章讲成“模型只能走到某些地方，我们在这些地方选离读数最近的点”，就已经抓住主线。矩阵公式是让同一个想法能处理更多读数和参数。

第四章　如何可靠地求解线性问题

想象两家店的总价记录：第一天买 1 个面包和 1 杯牛奶，第二天又买几乎相同的组合。总价可以测得很准，但要分别倒推出面包和牛奶的单价，就很容易受一分钱差异影响。要分清单价，最好再买一份明显不同的组合。

拟合也会遇到这种情况：**曲线看起来贴得很好，参数却可能很不稳**。电脑没有无限位的小数，它的计算误差又可能把问题放大。本章先认出这种危险，再认识可靠的计算工具。

4.1　先认识条件数：小误差会被放大多少

条件数描述问题对小扰动有多敏感。条件数很大，好比把几乎相同的购物组合拿来拆单价：读数稍变，算出的单价可能大变。它是一种风险提示，不是每次实际误差的精确倍数，也不是某个参数的置信区间。

下面的时间都挤在 1 附近，我们却要分清 $1, t, t^2, t^3, t^4$ 五种变化。它们在这个小区间很像，难以区分。先看 `condition number`，再比较 `parameter error` 的三个数：正规方程、QR、SVD 的参数误差。

图上 `singular value` 叫**奇异值**：它描述某种参数组合变化能在观测中留下多明显的信号。最小的特别小，说明至少有一种组合不容易从数据中分辨。纵轴用了对数刻度，每跨一个大格表示倍数变化，不是简单加一。

4.2　想知道计算工具做了什么

QR 可以理解成先换一套互相垂直、较好计算的坐标，再逐个求参数。薄 QR 分解 $A = QR$，其中 $Q^TQ = I$，把问题变成上三角方程 $Rx = Q^Ty$。“上三角” 表示表格左下方为零，可以从最后一项开始往回算。

SVD 把问题按 “哪些参数组合看得清、哪些看不清” 分开处理：

$$A = U\Sigma V^T, \qquad x^* = \sum_{\sigma_j > \tau} \frac{u_j^T y}{\sigma_j} v_j.$$

σ_j 是各方向的奇异值，v_j 是参数方向，u_j 是相应的观测方向。除以很小的 σ_j 会放大误差；τ 是用来判断 “小到不值得相信” 的阈值。第一次不必手算分解，会查看 `lstsq` 返回的秩和奇异值就够用。

正规方程为什么更危险？满列秩时，$\kappa_2(A^TA) = \kappa_2(A)^2$。这里 κ_2 是条件数。例如 A 已到 10^8，

形成 A^TA 后可到 10^{16}，接近双精度计算的精度极限。QR/SVD 避免这一步额外放大，但它们也不能让缺失的信息凭空出现。

```
t = 1 + .02*np.linspace(-1, 1, 40)
A = np.vander(t, 5, increasing=True)
truth = np.array([1., -2, 3, -.5, .2]); y = A@truth
Q, R = np.linalg.qr(A, mode='reduced')
x_qr = np.linalg.solve(R, Q.T@y)
x_svd, _, rank, s = np.linalg.lstsq(A, y, rcond=None)
try:
    x_normal = np.linalg.solve(A.T@A, A.T@y)
    normal_error = np.linalg.norm(x_normal-truth)
except np.linalg.LinAlgError:
    normal_error = np.inf
print('condition number:', np.linalg.cond(A), 'rank:', rank)
print('parameter error, normal / QR / SVD:', normal_error,
      np.linalg.norm(x_qr-truth), np.linalg.norm(x_svd-truth))
plt.semilogy(np.arange(1, len(s)+1), s, 'o-')
plt.xlabel('singular direction'); plt.ylabel('singular value'); plt.show()
```

```
condition number: 1400664882.8835294 rank: 5
parameter error, normal / QR / SVD: 2.537145251330903 9.232546032182727e-08 2.271433815690113e-08
```

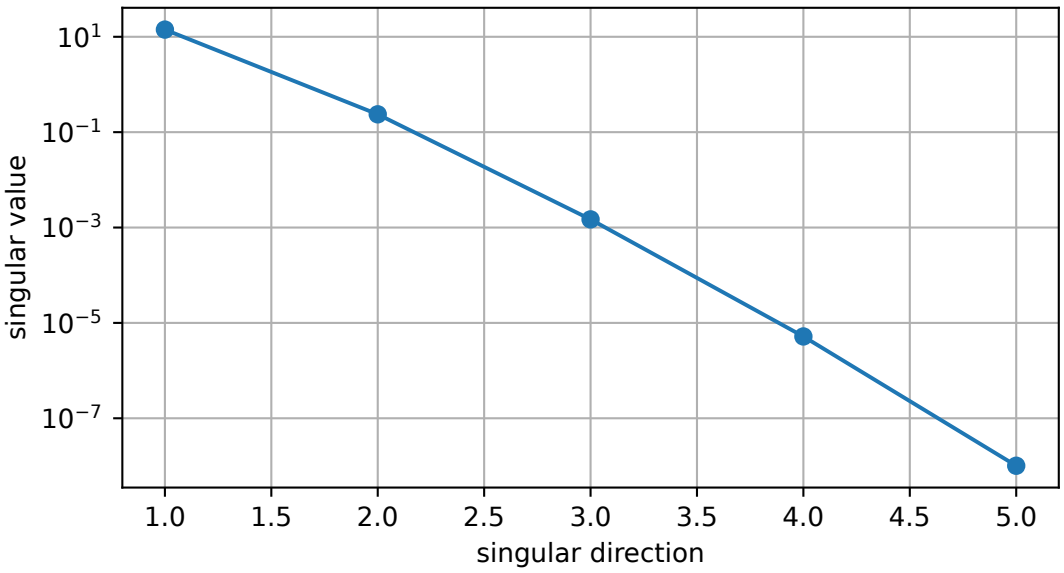


图 4.1: 病态设计矩阵的奇异值跨度

4.3 别把某台电脑的最后几位当成定理

不同底层计算库可能让误差大小不同，甚至让正规方程直接失败。真正要观察的是：数据很难区分参数时，计算方式很重要；**残差小和参数准不是同一个判断**。

再看一个更直接的例子。若模型永远只用两数的和 $x_1 + x_2$，那么 $(1, 1)$ 和 $(6, -4)$ 都预测 2。

下面故意把计算表的两列做成完全相同。`rank` 应为 1，说明两列只提供了一种独立信息。

SVD 通常会挑出 `[1, 1]`。这叫**最小范数解**：在同样好的参数里，选平方和最小的一组。它是一个选代表的规则，不能证明两参数的真实值就是各 1。

```
A = np.ones((4, 2)); y = np.array([1., 2., 2., 3.])
x, residual_summary, rank, s = np.linalg.lstsq(A, y, rcond=None)
print('minimum-norm x:', x, 'rank:', rank, 'summary:', residual_summary)
print('explicit squared residual:', np.sum((A@x-y)**2))
assert np.allclose(x, [1, 1])
assert np.allclose(A@x, A@(x+np.array([5., -5.])))
```

```
minimum-norm x: [1. 1.] rank: 1 summary: []
explicit squared residual: 2.0
```

4.4 一个常见误读：空结果不等于没误差

`lstsq` 返回的 `residuals` 在秩亏或观测数不多于参数数时可能是空数组。那是接口没有给这个摘要，不是所有误差为零。直接算 `A @ x - y` 才知道每次偏差，平方后求和才是总分。本例显式平方误差为 2。

4.5 动手试：把挤在一起的刻度拉开

在第一个实验中，把构造时间的范围改大，或者先计算 `u = (t - t.mean()) / .02`，再用 `u` 构造多项式表。观察条件数怎样变。这里**中心化**是先减平均值，**缩放**是再除一个合适尺度，使数值不挤在很窄的位置。

提示：换坐标后，同一条曲线的系数含义也变了。若要比较参数误差，必须把 `truth` 和造数据方式一起换到同一坐标；直接用旧系数作答案会比较错对象。初学时先比较条件数和预测曲线。

问题：最小范数解一定最符合物理吗？

解答：不一定。把某个参数的单位从米改成毫米，它的数值会放大一千倍，“参数最短” 的偏好也随之改变。如果你确实知道某个参数不该太大或必须非负，要把依据写成明确约束，第 06 章会接着讨论。

离开本章时记住一个动作：线性拟合先用 `np.linalg.lstsq`，同时看秩、奇异值和实际残差，不要只取返回的第一项就宣布测准了。

第五章　噪声、权重与估计可信度

同一辆小车，你有两种位置读数：一段来自较稳定的传感器，另一段抖动很大。把两段一视同仁，好像让模糊照片和清晰照片对细节有同样大的发言权。我们希望利用所有有用读数，同时承认它们的可靠程度不同。

5.1　先用白话说权重

权重决定某次读数在总分里占多大影响。常见做法不是凭喜好投票，而是先估计该仪器通常波动多大，再用“残差除以波动尺度”评分。

例如，残差同为 0.2，标准差为 0.1 时偏了两个尺度，标准差为 1 时只偏了五分之一个尺度。前者更值得让模型认真纠正。**标准差不是最大误差边界**，读数偶尔可以偏出它好几倍。

下面前半段标准差是 `.12`，后半段是 `.9`。图上的竖线表示一个标准差，长竖线表示更不稳定。`OLS` 是等权拟合，`WLS` 是按可靠程度加权。先猜后半段抖得更大时，哪条线会更照顾前半段，再运行看看。

5.2　想知道公式怎么写

若第 i 条计算表的预测是 $a_i^T x$，使用

$$F(x)=\frac{1}{2}\sum_i\left(\frac{a_i^T x-y_i}{\sigma_i}\right)^2.$$

展开后，平方误差前的权重为 $w_i=1/\sigma_i^2$。所以实际缩放残差和计算表时，除以 σ_i，而不是再除以 σ_i^2。代码中 `A/sigma[:, None]` 是给每行除自己的尺度。

若误差独立、呈高斯分布，也就是常见的钟形波动，并且尺度已知，这种办法还是**最大似然估计**：挑出让现有读数最不意外的参数。但不满足高斯假设，并不意味着不能计算最小二乘；失去的是那层特定的概率解释。

```
rng = np.random.default_rng(105)
t = np.linspace(0, 5, 40); A = np.column_stack([np.ones_like(t), t])
sigma = np.where(t < 2.5, .12, .9)
truth = np.array([1., 2.]); y = A@truth+rng.normal(size=len(t))*sigma
x_ols = np.linalg.lstsq(A, y, rcond=None)[0]
Aw, yw = A/sigma[:, None], y/sigma
```

```
x_wls = np.linalg.lstsq(Aw, yw, rcond=None)[0]
print('OLS:', x_ols, 'WLS:', x_wls)
plt.errorbar(t, y, yerr=sigma, fmt='.', alpha=.6, label='one sigma')
plt.plot(t, A@x_ols, label='OLS'); plt.plot(t, A@x_wls, label='WLS')
plt.xlabel('t'); plt.ylabel('response'); plt.legend(); plt.show()
```

```
OLS: [1.02546 1.97647] WLS: [1.12019 1.90625]
```

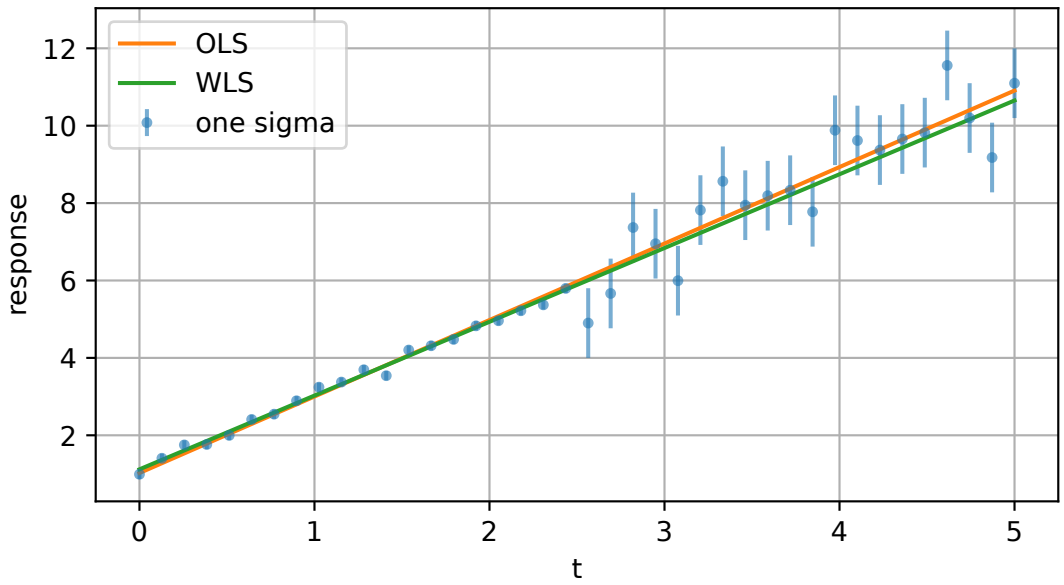


图 5.1: 异方差观测下的普通与加权拟合

5.3 几次读数可能一起错，不能当成几份新证据

如果同一个电源波动让十个传感器同时偏高，读了十次也不像十次互相独立的测量。**相关误差**就是误差会一起变化；**协方差矩阵**是记录各自波动和共同变化的一张表。

白化是一种换算：既调平不同读数的波动大小，也处理它们的共同变化，使新误差在协方差意义下不相关、尺度为 1。它不是洗掉数据或删异常点。本章代码用对角协方差演示，因此应与逐项除以标准差得到相同结果；一般相关的情形遵循同一公式，但不只逐行除一个数。

需要推导时：把正定协方差分解为 $C = LL^T$，用三角求解得到 $\tilde{A} = L^{-1}A$、$\tilde{y} = L^{-1}y$，因为

$$\|L^{-1}(Ax - y)\|^2 = (Ax - y)^T C^{-1}(Ax - y).$$

L^{-1} 表示解方程得到变换结果；实现不必真的求逆。方向不能误写成 L^{-T}。这里的“正定”保证这张误差表没有完全重复、零波动的方向。

5.4　参数也会随读数波动

用同一套真参数重新做 600 次有噪声的实验，每次拟合都会得到略不同的起点和速度。下面每个点就是一次实验的两个拟合参数；红叉是造数据时的真实设置。**参数标准误**描述这些估计通常会散多开，和原始读数的标准差不是一回事。

在线性模型正确、满列秩、观测协方差 C 已知时，参数协方差为

$$\operatorname{Cov}(\hat{x}) = (A^{T} C^{-1} A)^{-1}.$$

它的对角线是各参数的方差，开平方就是标准误。若只知道共同但未知的噪声大小，才常用 $\hat{\sigma}^2 = \|r\|^2/(m-n)$ 估计它，要求 $m > n$ 且满秩。$m-n$ 叫自由度：用了 n 个参数以后，剩下可用于估噪声的信息量。已知绝对噪声时，不要再无条件乘一次这个估计尺度。

```
C = np.diag(sigma**2)
Aw2, yw2 = whiten(A, y, C)
assert np.allclose(Aw2, Aw)
cov_x = np.linalg.solve(Aw.T@Aw, np.eye(2))
draws = np.array([np.linalg.lstsq(Aw, (A@truth+rng.normal(size=len(t))*sigma)/sigma,
                 rcond=None)[0] for _ in range(600)])
print('predicted standard errors:', np.sqrt(np.diag(cov_x)))
print('Monte Carlo standard errors:', draws.std(axis=0, ddof=1))
plt.scatter(draws[:, 0], draws[:, 1], s=5, alpha=.3)
plt.plot(*truth, 'rx', markersize=10); plt.xlabel('intercept'); plt.ylabel('slope')
plt.show()
```

```
predicted standard errors: [0.04918 0.03276]
Monte Carlo standard errors: [0.04694 0.03247]
```

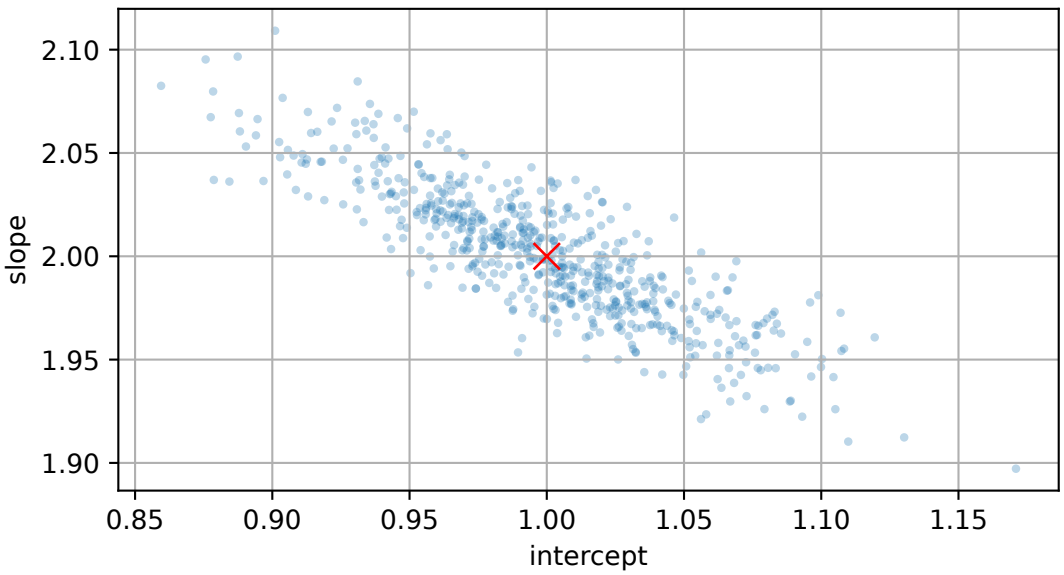


图 5.2: 重复仿真中的截距和斜率估计分布

5.5 看懂散点云，不只看两个误差条

横轴 `intercept` 是起点，纵轴 `slope` 是速度。点云倾斜，说明把起点估低一点、速度估高一点，仍可能解释同一批读数。因此两个参数的误差会互相配合，不是各自独立乱动。

`predicted standard errors` 是公式预测的散布，`Monte Carlo standard errors` 是 600 次实验数出来的散布。**蒙特卡洛**就是重复随机模拟来观察统计结果；次数有限，两者不该被要求一模一样。

常见的 $\hat{x}_j \pm 1.96\,\mathrm{SE}_j$ 是正态近似、已知协方差下单个参数的 95% 区间。意思是：按同一流程反复做实验，构造的区间在相应条件下约有 95% 覆盖固定真值。它不保证这一次一定覆盖，也不保证多个参数同时覆盖。未知尺度的线性高斯模型应改用 t 分布；遇到非线性、边界或信息不足，还需更谨慎。

5.6 动手试与解答

问题：只在拟合时把所有假定的 `sigma` 乘 2，保留已经生成的同一组 `y`，参数会变吗?

提示：所有人的分数一起除以 4，排名会变吗?

解答：参数不变，计算出的参数协方差乘 4，标准误乘 2。注意不要重新生成更吵的数据；那会同时改了另一个条件。

问题：两条完全相同、噪声也完全同步的观测，是双倍信息吗?

解答：不是。好比把同一张模糊照片复印一份，没有多拍一次。此时协方差有完全重复的方向，不能直接套本章要求正定的白化，需要合并重复信息或专门处理。

第六章　正则化与过拟合

你请人根据十几次小车位置画一条路线。第一人画得很弯，每个测量点都照顾到；第二人画得较平顺，允许离个别点有一点距离。新测一个位置时，第一条路线却可能偏得更离谱。

这叫**过拟合**：把测量中的随机起伏也当成了真实规律，像背熟练习题答案却不会做新题。问题不是“复杂一定错”，而是数据是否足以支持这么复杂的解释。

6.1　给“别为了追点而乱拐”一个明确规则

正则化就是在原来的误差分数之外，再为某种不希望出现的行为加分。本章惩罚的是过大的多项式系数，不是直接惩罚曲线的弯曲程度；它常能减轻乱摆，但效果与坐标和单位有关。

惩罚强度叫 λ：太小可能仍追着噪声跑；太大又可能把有用变化一起压掉。我们拿一批没参与拟合的读数比较，寻找一个合适程度，而不是凭曲线看起来顺眼来定。

下面用 14 阶多项式拟合 18 个点，故意让模型很灵活。左图 `OLS` 是不加惩罚，`ridge` 是加惩罚，虚线 `truth` 是生成模拟数据的规律。右图 `validation MSE` 是在另一批点上的平均平方误差，越低越好；横轴是惩罚强度的对数刻度。图的纵轴只显示 -2 到 2，超出的摆动可能被裁掉，别因此误判曲线没有剧烈变化。

6.2　想知道公式怎么写

$$F_\lambda(x)=\frac{1}{2}\|Ax-y\|^2+\frac{\lambda}{2}\|Dx\|^2.$$

第一项为没贴住数据付分，第二项为不希望的参数设置付分。D 决定罚谁：$D=I$ 时都罚；本例用 $D=\mathrm{diag}(0,1,\dots,1)$，第一项截距不罚，其他系数受罚。`diag` 是只在对角线上放数的表。

也可以把它写成普通最小二乘能直接处理的大表：

$$\min_x \left\| \begin{bmatrix} A \\ \sqrt{\lambda}D \end{bmatrix} x - \begin{bmatrix} y \\ 0 \end{bmatrix} \right\|^2 .$$

新增几行是在表达“这些受罚组合最好接近零”，不是新增真实测量。正则化愿意牺牲一点贴合程度，换取对波动不那么敏感；数学上叫用偏差换稳定性。

```
rng = np.random.default_rng(106)
```

```
f = lambda t: np.sin(np.pi*t)
t = np.linspace(-1, 1, 18); y = f(t)+rng.normal(0, .18, len(t))
tv = rng.uniform(-1, 1, 70); yv = f(tv)+rng.normal(0, .18, len(tv))
degree = 14
design = lambda t: np.vander(t, degree+1, increasing=True)
A, V = design(t), design(tv)
D = np.diag(np.r_[0., np.ones(degree)])
def ridge(lam):
    return np.linalg.lstsq(np.vstack([A, np.sqrt(lam)*D]),
                           np.r_[y, np.zeros(degree+1)], rcond=None)[0]
lambdas = np.logspace(-7, 1, 45)
validation = np.array([np.mean((V@ridge(l)-yv)**2) for l in lambdas])
best = lambdas[np.argmin(validation)]
plain = np.linalg.lstsq(A, y, rcond=None)[0]; regularized = ridge(best)
grid = np.linspace(-1, 1, 300)
print('chosen lambda:', best, 'coefficient norms:', np.linalg.norm(plain),
      np.linalg.norm(regularized))
fig, ax = plt.subplots(1, 2, figsize=(7, 3))
ax[0].scatter(t, y, s=12); ax[0].plot(grid, design(grid)@plain, label='OLS')
ax[0].plot(grid, design(grid)@regularized, label='ridge')
ax[0].plot(grid, f(grid), '--', label='truth'); ax[0].set_ylim(-2, 2)
ax[0].set(xlabel='t', ylabel='response'); ax[0].legend()
ax[1].semilogx(lambdas, validation); ax[1].set(xlabel='lambda', ylabel='validation MSE')
plt.tight_layout(); plt.show()
```

```
chosen lambda: 0.00043287612810830614 coefficient norms: 3761.5230070368407 6.291813881254787
```

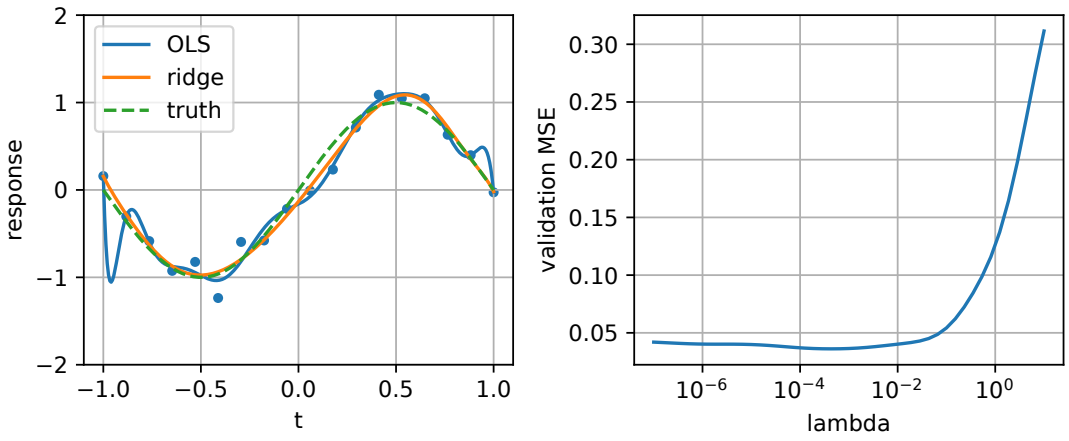


图 6.1: 正则化曲线与独立验证误差

6.3 三批数据，做三份不同的工作

训练集用来调曲线参数；**验证集**用来选 λ 等设置；**测试集**最后才拿出来检查。像做题、根据模拟考改学习策略、最后正式考试。如果看过考试答案再改方案，就不能拿同一成绩当作全新考试的表现。

上面用独立验证数据选强度。下面再生成第三批数据，比较两个方案在新题上的 MSE，即残差平方的平均值。它是一次随机实验的结果，不是保证岭型方法在所有数据上都赢。

6.4 进阶解释：它为什么能压住不稳定的方向

当 $D = I$，SVD 中原来的放大系数 $1/\sigma_j$ 变为 $\sigma_j/(\sigma_j^2 + \lambda)$。对很小的奇异值，不再盲目地除以一个很小的数，误差就不容易被放大。

在高斯噪声和高斯先验下，它还可解释成**最大后验估计**：同时照顾数据证据，以及测量前对参数的合理预期。若观测噪声方差是 σ^2、零均值参数先验方差是 τ^2，在未白化的上述目标中 $\lambda = \sigma^2/\tau^2$。这是带条件的解释，不是所有惩罚都等于这种先验。参数单位变了，惩罚也要相应调整。

```
tt = rng.uniform(-1, 1, 200)
yt = f(tt)+rng.normal(0, .18, len(tt))
print('independent test MSE, OLS:', np.mean((design(tt)@plain-yt)**2))
print('independent test MSE, ridge:', np.mean((design(tt)@regularized-yt)**2))
assert np.all(np.isfinite(regularized))
```

```
independent test MSE, OLS: 0.06030827732864676
independent test MSE, ridge: 0.03305857997927622
```

6.5 动手试：给曲线松绑或收紧

先看右图误差最低处，再试把 best 手动改成 1e-7 和 10.，分别重算 regularized 并画图。每次从造数据处重新运行，保持种子不变，保证比较的是同一组题。

先猜：很大的惩罚，会让线接近零，还是接近一条水平线？

解答：本例不罚截距，其他系数被压向零，所以更接近水平线，位置不必为零。如果把截距也罚进去，结果又不同。这就是为什么要说清“罚什么”，不能只报一个 λ。

问题：多加一个参数，带正则化结果的训练残差一定更低吗？

解答：不一定。优化时在平衡“贴数据”和“参数费用”两个分数。第 03 章训练误差不升的结论，针对的是不另加惩罚的普通最小二乘。

问题：第 10 章 LM 也会加一个类似的项，和本章一样吗？

解答：本章限制最终参数，改变想找的答案；LM 主要限制这一次怎么走，通常不会把这笔步长费用加入最终评分。一个是改变目的，一个是调节走法。

第七章　进入非线性最小二乘

你想调一个有三个旋钮的装置：一个管起始强度，一个管下降速度，一个管背景。现在曲线没贴住读数，该先拧哪个旋钮、拧多少？

线性问题像刻度均匀的旋钮，作用可以一次算清。非线性问题里，旋钮的作用会随当前设置变化。我们先做一个小范围试探：**在当前位置，每个旋钮稍动一点，曲线大约会怎么变？**先用这张临时地图走一小步，再重新量地图，而不是一次猜到终点。

7.1　雅可比：一张“旋钮影响表”

名字有点陌生，意思很具体。**雅可比矩阵**的每行对应一次观测，每列对应一个参数，格子里放“这个参数增加一点，那个残差大约增加多少”。正数表示同向，负数表示反向，接近零表示这里不太受它影响。

还是模型 $f(t) = a\exp(-kt) + c$：调大 a 会抬高早期响应；调大正的 k 会使正幅值曲线下降更快；调大 c 则把整条曲线抬高。参数顺序固定为 $x = (a, k, c)^T$ 时，第 i 行是

$$J_i = \begin{bmatrix} e^{-kt_i} & -at_ie^{-kt_i} & 1 \end{bmatrix}.$$

这些格子是**偏导数**，也就是“先不动其他旋钮，只看这个旋钮的变化率”。当残差是模型减观测、观测固定时，模型和残差的影响表相同。

下面把手写的 `jac` 和通过小扰动测出的 `finite_difference` 对比。应看到 `Jacobian shape: (30, 3)`，最大导数差异很小。先不用追求每一位相同；关注二者是否在允许精度内一致。

7.2　想知道公式怎么写：局部线性化

把这次参数改变量记作 h。足够小的改动下，

$$r(x + h) \approx r(x) + J(x)h.$$

这叫**局部线性化**：把弯曲规律的一小段暂时当成直的。$\approx$ 是“大约等于”，不是保证远处也准确。于是当前小步的评分可用 $q(h) = \frac{1}{2}\|r + Jh\|^2$ 近似，而这又成了一个线性最小二乘问题。

进阶时还会用到两份变化记录：

$$g = J^T r, \qquad H = J^T J + \sum_{i=1}^{m} r_i \, \nabla^2 r_i.$$

g 叫**梯度**，记录各参数怎样影响总分；H 叫 **Hessian（海森矩阵）**，记录这些变化率又怎样改变，也就是曲率。第二项是每个残差本身的弯曲贡献。残差小、二阶导数不过大时它可能较小；残差大时不能随意丢掉。后面的 GN 和牛顿法，区别之一就在怎样利用这些信息。

```
t = np.linspace(0, 4, 30)
p = np.array([2.5, .8, .4])
def model(p):
    a, k, c = p
    return a*np.exp(-k*t)+c
def jac(p):
    a, k, c = p
    e = np.exp(-k*t)
    return np.column_stack([e, -a*t*e, np.ones_like(t)])
J = jac(p)
numerical = finite_difference(model, p)
print('Jacobian shape:', J.shape)
print('largest derivative difference:', np.max(np.abs(J-numerical)))
assert np.allclose(J, numerical, rtol=1e-7, atol=1e-8)
direction = np.array([.3, -.2, .1])
steps = np.logspace(-5, -1, 12)
errors = [np.linalg.norm(model(p+h*direction)-model(p)-h*J@direction) for h in steps]
plt.loglog(steps, errors, 'o-', label='linearization error')
plt.loglog(steps, errors[-1]*(steps/steps[-1])**2, '--', label='quadratic slope')
plt.xlabel('step length'); plt.ylabel('error norm'); plt.legend(); plt.show()
```

```
Jacobian shape: (30, 3)
largest derivative difference: 5.2015725060527984e-11
```

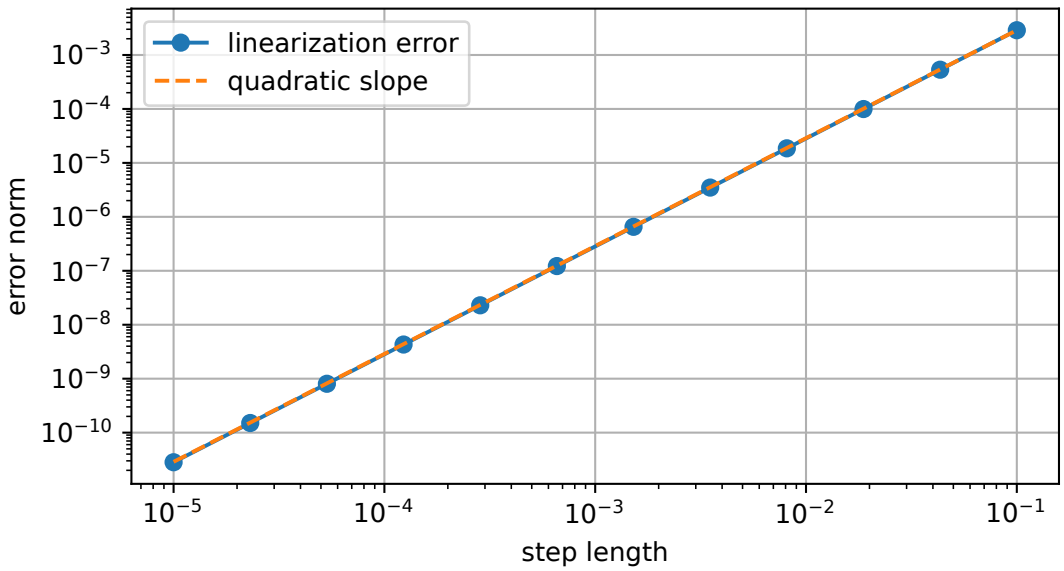


图 7.1: 局部线性化误差随步长变化

7.3 怎么读刚才的图

横轴 `step length` 是试探的步子，纵轴 `error norm` 是临时直线预测与真正曲线之间的差距。两根轴都是对数刻度。实线若大致跟虚线 `quadratic slope` 平行，表示在这个范围内，步子缩小十倍，近似误差大约缩小一百倍。它支持的是**附近可以这样近似**，不是说整个问题已经解好了。

7.4 不会求导，也可以试着量；但别把步子缩到看不见

中心差分就是先向右拧一点，再向左拧一点，比较结果：

$$J_{:,j} \approx \frac{r(x + \delta_j e_j) - r(x - \delta_j e_j)}{2\delta_j}.$$

e_j 表示只动第 j 个参数，δ_j 是试探幅度。分子是左右两次读数差，分母是两次设置相隔的距离。

步子太大，弯曲效应混进来了；太小，两个电脑小数几乎一样，舍入误差会占上风。光滑、无测量噪声时，中心差分的近似误差通常按 δ_j^2 缩小，写作 $O(\delta_j^2)$。代码用 $\delta_j = \epsilon^{1/3}\max(1, |x_j|)$ 作为双精度的起点，ϵ 是机器精度。这个经验值不是仪器实验的通用答案。

真实仪器还有抖动。两次测量之差再除很小的幅度，会把抖动放大。像用一把会晃的尺子去量一根头发，再放大读数，未必更准。第 15 章会讨论有效扰动和重复测量。

7.5 自己试与核对

先把 `steps` 的最小端从 `1e-5` 扩到 `1e-12`，重新看图。你可能看到小步端不再遵循原来的斜率。准确拐点因机器而异，不要把某个固定步长记成绝对界限。

问题一：若 $a = 0$，还能从附近观测判断 k 吗？

解答：不能靠这个局部影响表判断。指数部分的幅度为零，调下降速度也不改变输出，第二列全零。这是缺少信息，换一个更响亮的算法名字没用。

问题二：把残差和雅可比一起改符号，$g = J^T r$ 会变吗？

解答：不会，两个负号相消。

问题三：梯度为零，是否已经找到全局最低分？

解答：没有。它只表示当前位置一阶变化为零，像站在平坦处，也可能是山顶或马鞍形地形。全局最低还需要更多判断。

第八章　迭代优化的基本工具

闭着眼下山时，你可以先摸出哪个方向往下，再决定迈多大一步。方向大致对，不代表大跨步也安全。非线性拟合每轮也在做这两件事：**往哪儿调，以及这次调多少**。

8.1　先找下坡，再试脚下

梯度指向总分上升最快的局部方向。沿负梯度走，叫**最速下降**。这里的“最速”只管脚下那一点、同样长度的小步，不表示整个旅程最快。

选好方向后，先试完整一步。如果评分没按预期变好，就缩短一半，再试一次。这个过程叫**回溯线搜索**：方向先不改，只把步子收小。它要有尝试上限，不能无限试下去。

下面图里的等高线像地形图，同一条线上总分相同。点线是逐次调整的轨迹，红叉是这个例子的最低点。先猜：在狭长的山谷里，直朝脚下最陡方向走，会不会左右来回摆？

8.2　想知道判断标准：不能只是碰巧降了一点

把当前梯度记为 g、方向记为 h。若 $g^Th<0$，这个方向足够小的正步长会下降。回溯中的 Armijo 条件是

$$F(x+\alpha h)\le F(x)+c\alpha g^Th,\qquad 0<c<1.$$

α 管步子长度，c 是要求下降程度的小常数。右边比当前分数低，表示“实际改善至少达到局部预测的一定份额”。本例用 $c=10^{-4}$，初始 $\alpha=1$，不合格就减半。

为什么会摆动？对 $F(x)=\frac12x^TBx$，某些方向特别陡、某些特别缓，等高线就狭长。固定步长稳定需要 $0<\alpha<2/\lambda_{\max}(B)$，其中 $\lambda_{\max}$ 描述最陡的曲率。为了在陡方向不冲过头而选的小步，在缓方向就显得很慢。

```
B = np.array([[10., 8.], [8., 10.]])
cost = lambda x: .5*x@B@x
x = np.array([3., 0.]); path = [x.copy()]
for _ in range(35):
    g = B@x; h = -g; alpha = 1.
    while cost(x+alpha*h) > cost(x)+1e-4*alpha*(g@h):
        alpha *= .5
    x = x+alpha*h
```

```
    path.append(x.copy())
path = np.array(path)
u, v = np.meshgrid(np.linspace(-1, 3.2, 120), np.linspace(-2, 1.2, 120))
z = .5*(B[0,0]*u*u+2*B[0,1]*u*v+B[1,1]*v*v)
plt.contour(u, v, z, levels=18)
plt.plot(path[:,0], path[:,1], 'o-', markersize=2, label='gradient + backtracking')
plt.plot(0,0,'rx',label='Newton solution')
plt.xlabel('x1'); plt.ylabel('x2'); plt.legend(); plt.show()
print('gradient method final norm:', np.linalg.norm(x))
```

```
gradient method final norm: 2.0236746917870024e-06
```

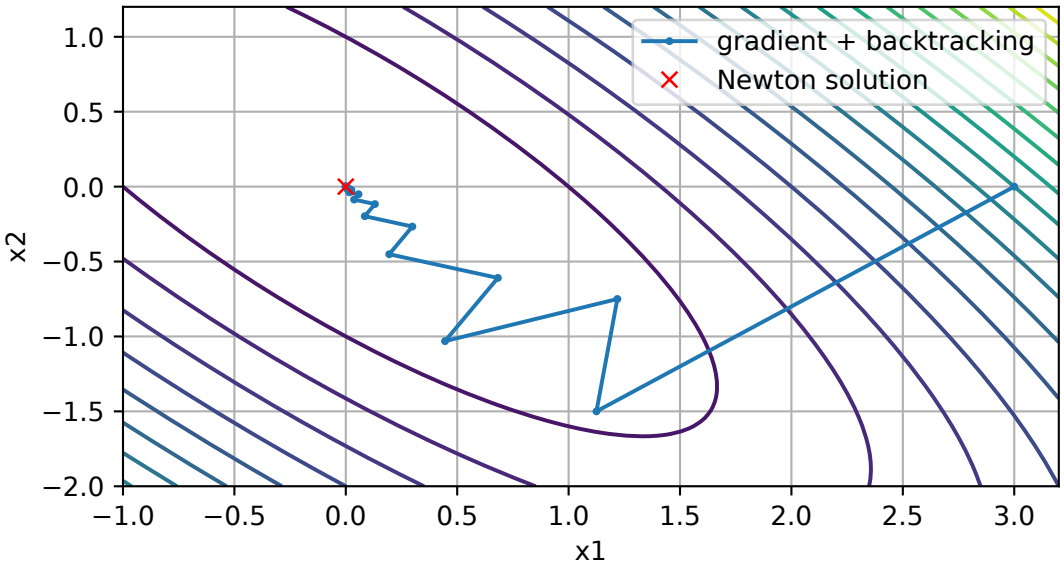


图 8.1: 狭长二次目标上的梯度回溯路径

8.3 牛顿法：不仅问往哪儿降，还估计地形怎样弯

刚才的路线可能左右摆动。**牛顿法**进一步利用曲率，预测附近碗底在哪里。曲率由 Hessian 矩阵 H 记录，更新方向满足

$$Hh = -g.$$

它来自二阶近似 $F(x+h) \approx F(x) + g^T h + \frac{1}{2} h^T H h$。当附近像朝上的碗、$H$ 正定时，牛顿方向是下降方向；对精确正定二次函数可以一步到碗底。但如果附近更像山顶，照着“找平处”的方程走，反而可能上山。

下面故意用 $F(z) = z^4 - 3z^2$，从 `z = .3` 出发。看打印的前后分数：更负的数才更低；即使两个值都为负，也要认真比较。`g*h` 为正、完整一步后分数更高，就是这个牛顿方向不下降的证据。**多算二阶信息，不等于从此不用检查结果**。

```
z = .3
```

```
g = 4*z**3-6*z
H = 12*z**2-6
h = -g/H
F = lambda z: z**4-3*z**2
print('Hessian:', H, 'g*h:', g*h)
print('cost before/after full Newton step:', F(z), F(z+h))
assert g*h > 0
assert F(z+h) > F(z)
```

```
Hessian: -4.92 g*h: 0.5818829268292681
cost before/after full Newton step: -0.2619 -0.005778557493008074
```

8.4 程序停下了，可能有几种完全不同的意思

现象	白话解释	还要看什么
梯度很小	眼前看不出明显下降方向	是否局部停点、参数是否分得清
步子很小	每轮几乎没移动	是真靠近答案，还是被阻尼压住或受数值精度限制
分数不再明显降低	这几次调整改善很少	是否噪声下限、模型不对或走法失败
次数用完	预算到了	不能把它改叫成功

本书求解器会返回参数、分数、停止状态和历史。历史不是为了凑图，而是让你能复查“它到底走过什么”。即使满足停止条件，也还要检查物理范围和残差图。

8.5 动手试与解答

把第一个实验的 `B` 换成 `np.eye(2)`，让地形从狭长山谷变成圆碗，再运行。轨迹为什么不怎么摆了？

解答：各方向尺度一致。在这个特定二次问题里，合适的步子可直接到中心；原来的困难来自方向尺度差别，不是循环次数写得不够大。

再问：若方向本来就朝上，不断缩短步子一定能救回来吗？

解答：不能保证。足够小的正步仍可能上升，应先修正方向。

“牛顿法二次收敛”说的是在光滑性、解附近 Hessian 非奇异等条件满足时，已经靠近答案后误差可非常快地缩小。它没有承诺任意起点都能走到那里。

第九章　Gauss–Newton 方法

第 07 章已经量出了“旋钮影响表”。现在不妨这样做：先假装这张表在附近准确，算一组最合适的小调整；走过去后，重新量表，再算下一步。这就是 **Gauss–Newton（高斯—牛顿，简称 GN）方法**。

它像不断更新的小地图，不是拿一张平地图走遍整座山。主线只有三步：看当前误差、按局部直线规律求调整、到新位置再检查。

9.1　先看它会犯什么错

下面用一个只有两个参数的弯谷例子。我们知道真正最低点在 $(1,1)$，从 $(-1.2,1)$ 出发，先故意不限制步子。`unprotected full-step costs` 的第一项是起点分数，第二项是第一步后的分数。先猜：第一步会不会反而更糟？

`full GN` 表示直接走完局部建议，`GN + backtracking` 表示走前先试、太远就缩小。纵轴 `cost` 是总误差分数，越低越好；用了对数刻度，不要把图上的竖直距离当作普通差值。

这个例子的残差写作 $r=(10(x_2-x_1^2),1-x_1)^T$，叫 Rosenbrock 型残差。记住名字没有必要；它只是一个能让“大步冲过头”显形的小型测试地形。

9.2　想知道公式怎么来

把临时线性残差 $r+Jh$ 的平方和最小化：

$$h_{\mathrm{GN}}=\arg\min_h \|Jh+r\|^2,\qquad x_{\mathrm{new}}=x+h_{\mathrm{GN}}.$$

h 是这轮改变量，不是最终参数。满列秩时它满足 $J^TJh=-J^Tr$，但代码直接用 `lstsq(J, -r)` 求解，避免先形成正规方程。

这相当于拿 J^TJ 近似 Hessian，省掉第 07 章的 $\sum_i r_i\nabla^2 r_i$。残差接近零且导数行为良好时，这种近似往往很好；若残差大、起点差、曲线弯得厉害，就可能失准。

```
def rfun(x):
    return np.array([10*(x[1]-x[0]**2), 1-x[0]])
def jfun(x):
    return np.array([[-20*x[0], 10.], [-1., 0.]])
x = np.array([-1.2, 1.])
```

```
raw = [.5*rfun(x)@rfun(x)]
for _ in range(6):
    h = np.linalg.lstsq(jfun(x), -rfun(x), rcond=None)[0]
    x = x+h
    raw.append(.5*rfun(x)@rfun(x))
print('unprotected full-step costs:', np.array(raw))
assert raw[1] > raw[0]
safe = gauss_newton(rfun, jfun, [-1.2,1.])
print('safeguarded estimate/status:', safe.x, safe.status)
costs = [p['cost'] for p in safe.history]
plt.semilogy(np.maximum(raw, 1e-20), 'o-', label='full GN')
plt.semilogy(np.maximum(costs, 1e-20), '.-', label='GN + backtracking')
plt.xlabel('iteration'); plt.ylabel('cost'); plt.legend(); plt.show()
assert np.allclose(safe.x, [1,1], atol=1e-6)
```

```
unprotected full-step costs: [  12.1  1171.28    0.      0.      0.      0.      0.  ]
safeguarded estimate/status: [1. 1.] gradient
```

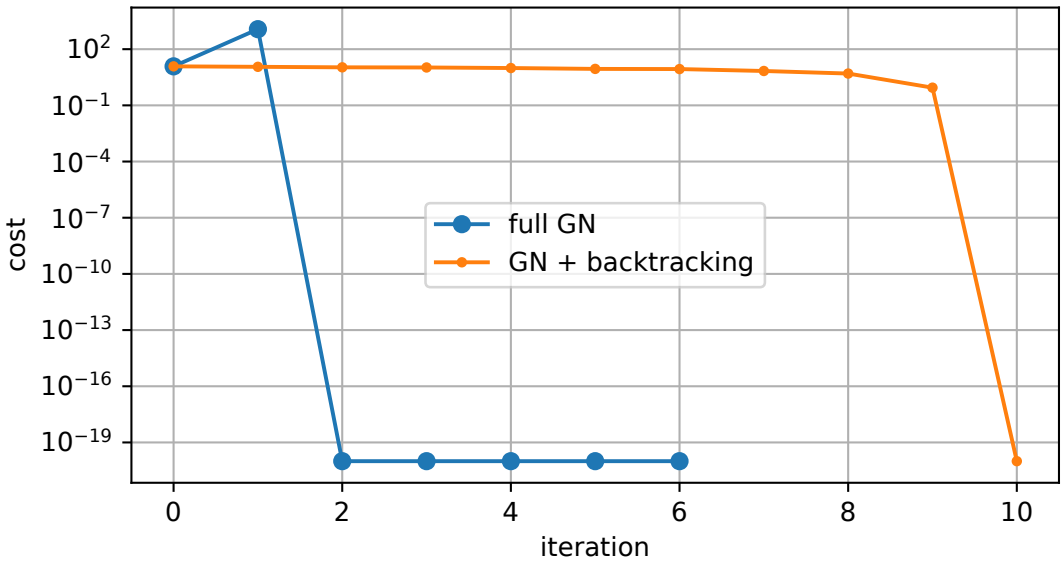


图 9.1: 未经保护与回溯 GN 的目标变化

9.3 给 GN 加一个“先试再走”的习惯

解出方向 h 后，不马上照单全收，而是先算真实的 $F(x+\alpha h)$，必要时将 α 减半。这就是上章的回溯保护。它承认局部地图有用，却不一定管得了远处。

下面回到下降曲线。造数据时加了噪声，所以即使找到合理参数，总误差也不会必然归零。`ours` 是本书带保护的 GN，`SciPy` 是成熟工具作参照。两组参数和分数应接近；图上的拟合线应抓住总体下降趋势，而不是硬穿每个点。

这里的“真实目标非零”是说**原评分函数在拟合结果处不为零**，不是说我们采集了真实仪器数据。本章仍是合成数据实验。

```
rng = np.random.default_rng(109)
t = np.linspace(0, 4, 40)
y = 2.5*np.exp(-.8*t)+.4+rng.normal(0,.035,len(t))
def residual(p):
    return p[0]*np.exp(-p[1]*t)+p[2]-y
def jac(p):
    e = np.exp(-p[1]*t)
    return np.column_stack([e,-p[0]*t*e,np.ones_like(t)])
ours = gauss_newton(residual, jac, [2., .5, .2], gtol=1e-8)
reference = least_squares(residual,[2., .5, .2],jac=jac,gtol=1e-12,ftol=1e-12,xtol=1e-12)
print('ours:', ours.x, ours.status, 'cost:', ours.cost)
print('SciPy:', reference.x, 'cost:', reference.cost)
assert np.allclose(ours.x, reference.x, atol=2e-5)
plt.scatter(t,y,s=12,label='data'); plt.plot(t,residual(ours.x)+y,label='fit')
plt.xlabel('time'); plt.ylabel('response'); plt.legend(); plt.show()
```

```
ours: [2.50686 0.80254 0.4034 ] gradient cost: 0.013858140938455986
SciPy: [2.50686 0.80254 0.4034 ] cost: 0.013858140938455986
```

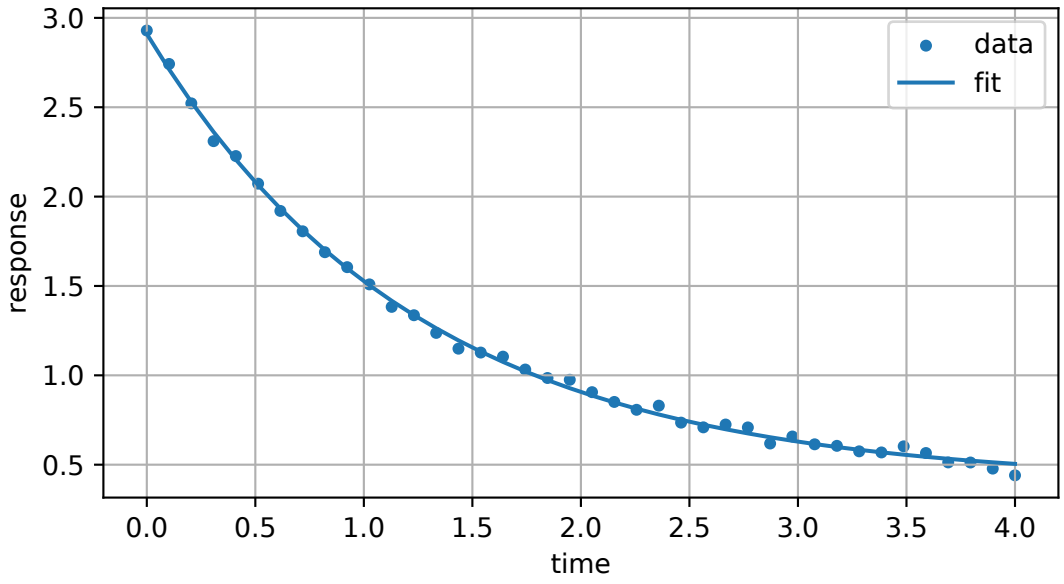


图 9.2: GN 拟合有噪声指数观测

9.4 卡住时，先问地图和数据出了什么问题

如果两列雅可比几乎在做同一件事，叫“近秩亏”，局部计算可能建议一个很大的步；如果参数数值相差几千倍，同样的步长含义可能很不一致。先改善单位、缩放和测量条件，再考虑 LM 或信赖域限制步子。

参数完全分不清时，保护走法也不能补出缺少的观测信息。像只知道总价，不能因为换一种计算器就拆出每件单价。

9.5 自己试一下

在第一个实验中，把两处起点一起改成 `[-.5, .5]`，比较有无回溯的曲线。原来的 `assert raw[1] > raw[0]` 只针对原起点；换起点探索时可以先注释它，别把“必须第一步变差”当成算法定律。

问题一：如果残差本来就是 $Ax - y$，GN 要走几次？

解答：精确计算、满秩且完整走步时，一次就到最小二乘解。因为临时地图恰好就是完整问题。

问题二：读数个数比参数少，也能返回一步吗？

解答：可以选一个最小范数步，但有些参数方向缺少约束，不能因此宣称答案唯一。

问题三：未经保护最后也到了最低点，为什么还要保护？

解答：中途的大幅变差在其他模型里可能产生溢出、负的物理量或跑到另一个局部最低点。本次最后成功，没有证明这种走法处处可靠。

第十章　Levenberg–Marquardt 方法

GN 有时提出一个过大的调整。我们不一定要把它整个丢掉，可以先加一句：“这次改动越大，就多付一点费用。”当局部地图不可靠时收紧，实际效果好时再放松。

这就是 **Levenberg–Marquardt 方法，简称 LM**。先记它的工作方式：**给步子装一个可调的刹车，再用实际结果判断刹得是否合适**。正式名称叫“阻尼”。它不像真实摩擦那样作用于物体，而是作用于计算出的参数改变量。

下面在同一位置试三种 `mu`。`step` 是建议步，`actual/predicted` 是实际改善与预期改善的比。先看大 `mu` 是否让步子更保守，再判断改善是不是跟着出现。别先认定“大阻尼必然成功”。

10.1　想知道公式怎么写：罚这一步，不是罚最终答案

$$\min_h \frac{1}{2}\|Jh+r\|^2+\frac{\mu}{2}\|h\|^2, \qquad (J^TJ+\mu I)h=-g.$$

第一项希望误差下降，第二项不鼓励这轮调得太猛。I 是单位矩阵，作用像“每个参数方向各记一份步长费用”。μ 越大，限制通常越强；很大时 $h\approx -g/\mu$，像小步负梯度；很小时接近 GN。

第 06 章正则化罚的是最终参数 x，会改变要找的答案；这里罚的是当前步 h，通常只用来安排走法，不加入最终原目标。参数单位也影响步长长度，因此仍需合理缩放。

实现把 J 下方接上 $\sqrt{\mu}I$，用增广最小二乘计算，而不是直接求 J^TJ 的逆。

```
def rfun(x):
    return np.array([10*(x[1]-x[0]**2), 1-x[0]])
def jfun(x):
    return np.array([[-20*x[0], 10.], [-1., 0.]])
x = np.array([-1.2,1.]); r = rfun(x); J = jfun(x)
for mu in (.001, 1., 100.):
    augmented = np.vstack([J,np.sqrt(mu)*np.eye(2)])
    h = np.linalg.lstsq(augmented,np.r_[-r,np.zeros(2)],rcond=None)[0]
    predicted = -(J.T@r)@h-.5*(J@h)@(J@h)
    actual = .5*(r@r-rfun(x+h)@rfun(x+h))
    print('mu, step, actual/predicted:',mu,h,actual/predicted)
```

```
mu, step, actual/predicted: 0.001 [ 2.18628 -4.80702] -93.40607323352172
mu, step, actual/predicted: 1.0 [ 0.42134 -0.56555] 0.8597119121338301
mu, step, actual/predicted: 100.0 [0.14139 0.05033] 0.9877615479733826
```

10.2　地图说能省 10 分，真的省了多少

临时模型预测下降 10 分，实际下降 8 分，比值就是 0.8；实际反而涨分，比值就为负。这叫**增益比** ρ，用来判断局部地图有没有兑现承诺。

局部预测下降来自原目标的模型：

$$q(0)-q(h)=-g^{T}h-\frac{1}{2}\|Jh\|^{2},$$

因此

$$\rho=\frac{F(x)-F(x+h)}{-g^{T}h-\frac{1}{2}\|Jh\|^{2}}.$$

分母必须为正；预测本来就没有改善时，不能直接相除再说模型可信。本书实现中，$\rho>0$ 才接受新参数；否则保留旧位置、加大阻尼。接受后再按比值调整阻尼。

拒绝一次尝试，不等于什么也没做。它仍消耗了一次或多次函数计算，只是没移动到新位置。下面打印 `rejected attempts`，就是记录这些试错成本。

左图是已经接受的位置所对应的分数，右图是刹车强度 `damping mu`。尝试后左图保持水平，可能是这次被拒绝；不是参数一定已经到了终点。

进阶核对：当阻尼方程被精确求解时，预测下降也等于 $\frac{1}{2}h^{T}(\mu h-g)$。上面的写法更直接地说明，它预测的是原评分的改善，不是包含刹车费用的新评分。

```
ours = levenberg_marquardt(rfun,jfun,[-1.2,1.],max_iter=200)
reference = least_squares(rfun,[-1.2,1.],jac=jfun,method='lm')
print('LM estimate/status:',ours.x,ours.status)
print('SciPy LM:',reference.x)
print('rejected attempts:',sum(not p['accepted'] for p in ours.history))
assert np.allclose(ours.x,reference.x,atol=1e-6)
history = ours.history[1:]
fig,ax = plt.subplots(1,2,figsize=(7,3))
ax[0].semilogy([max(p['cost'],1e-20) for p in ours.history],'.-')
ax[0].set(xlabel='attempt',ylabel='accepted-state cost')
ax[1].semilogy([p['control'] for p in history],'.-')
ax[1].set(xlabel='attempt',ylabel='damping mu')
plt.tight_layout();plt.show()
```

```
LM estimate/status: [1. 1.] gradient
SciPy LM: [1. 1.]
rejected attempts: 2
```

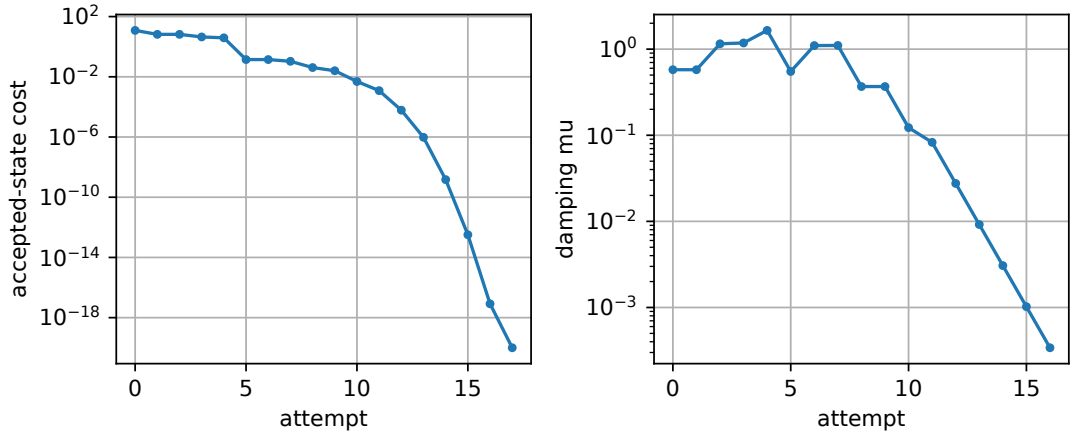


图 10.1: LM 的目标历史与阻尼历史

10.3 刹车有用，但不是自动驾驶的全部

LM 不能保证找到全局最低点，也不会自动满足参数上下界。要求速率非负时，应选支持边界的求解方式，例如第 12 章的 SciPy `trf`。若仪器在一段范围内拧了也没反应，雅可比为零，单纯增加阻尼也不会让信号出现。

本书如果发现阻尼很大、步子极小、梯度仍不小，会报 `stalled`，意思是“卡住了”，不是“已经成功”。图上尝试次数也不能直接拿来比较不同库的效率，它们对尝试和求值的计数未必相同。

10.4 动手试与解答

在第一格三种 `mu` 中加入 `10.` 和 `1000.`，比较步长和增益比。先猜它们的趋势，再看实际打印。这个小实验只比较同一位置的一步，不代表完整优化所需时间。

问题一：加大 μ，真实分数就一定下降吗？

解答：不一定。它主要让局部步保守，真正是否变好仍要算原目标检查。

问题二：拒绝一步后，当前位置变了吗？

解答：没有。调整阻尼，从原位置重新提议。

问题三：预测下降为零怎么办？

解答：先别相除。检查是否已到一阶停点、方向没信息或有数值问题，再明确停止或拒绝。返回状态应把理由告诉使用者。

第十一章　信赖域与狗腿法

LM 是“大步要额外交费”。还有一种更直接的规则：“附近我只摸清了一圈，这次最多走到圈边。”这圈范围叫**信赖域**，半径记作 Δ。信赖的是局部地图的适用范围，不是对最终参数有多大信心。

我们要在 $\|h\| \le \Delta$ 内，让临时预测的误差尽量小。圆是两个参数时的画法，更多参数时变成高维球。不同参数单位仍要处理，否则这个“距离”可能不公平。

11.1　狗腿法：沿一条折线路线选步

不在圈里到处搜索，而是先算两个候选：一个沿负梯度下坡，叫 **Cauchy（柯西）步**；另一个是 GN 建议的完整局部步。把“原地—柯西步—GN 步”连成折线，在这条折线上选圈内的点，因为拐了一下，叫**狗腿法**。

选择规则并不神秘：

1. GN 步已经在圈内，就直接采用。
2. 连柯西步都在圈外，就沿下坡方向走到圈边。
3. 柯西步在圈内、GN 步在圈外，就取后半段折线与圈边的交点。

下面展示第三种情况。虚线圆 `trust boundary` 是允许范围，`dogleg path` 是候选路线，红叉 `selected step` 是最终这一步。注意坐标是**改变量** `h1`，`h2`，不是最终参数值；原点表示这轮尚未移动。

11.2　想知道公式怎么来

沿负梯度最小化同一局部模型，可得

$$h_{\mathrm{C}} = -\frac{g^T g}{\|Jg\|^2} g.$$

这里假设非零梯度且分母有效；零梯度或退化情况要另作停止判断。第三种分支令 $v = h_{\mathrm{GN}} - h_{\mathrm{C}}$，在 $h = h_{\mathrm{C}} + \beta v$ 上求交点：

$$\|v\|^2 \beta^2 + 2 h_{\mathrm{C}}^T v\, \beta + \|h_{\mathrm{C}}\|^2 - \Delta^2 = 0.$$

β 是从柯西步走向 GN 步的比例，选 $0 \le \beta \le 1$ 的根。狗腿法是方便的近似路线，并非一般信赖域问题精确答案的通用公式。

```
x = np.array([-1.2,1.])
r = np.array([10*(x[1]-x[0]**2),1-x[0]])
J = np.array([[-20*x[0],10.],[-1.,0.]])
g = J.T@r
hc = -(g@g)/((J@g)@(J@g))*g
hgn = np.linalg.lstsq(J,-r,rcond=None)[0]
radius = 1.; v = hgn-hc
a,b,c = v@v,2*hc@v,hc@hc-radius**2
beta = (-b+np.sqrt(b*b-4*a*c))/(2*a)
h = hc+beta*v
assert 0 <= beta <= 1
assert np.isclose(np.linalg.norm(h),radius)
theta = np.linspace(0,2*np.pi,200)
plt.plot(radius*np.cos(theta),radius*np.sin(theta),'--',label='trust boundary')
plt.plot([0,hc[0],hgn[0]],[0,hc[1],hgn[1]],'o-',label='dogleg path')
plt.plot(*h,'rx',markersize=10,label='selected step')
plt.axis('equal');plt.xlabel('h1');plt.ylabel('h2');plt.legend();plt.show()
```

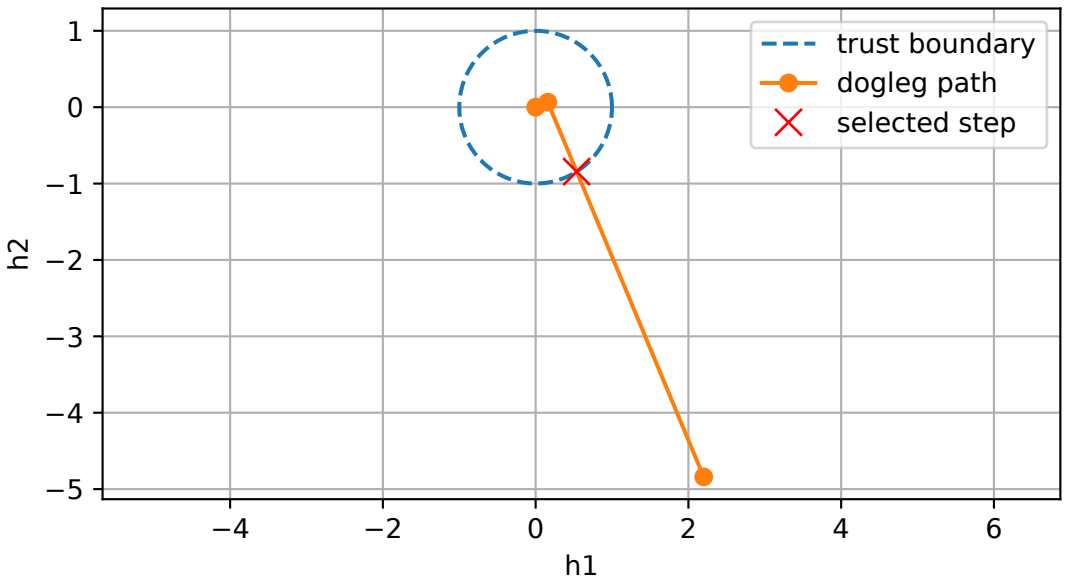


图 11.1: 狗腿折线与信赖域球面交点

11.3 一圈画多大，要听实际结果

仍用“实际下降除以预测下降”的增益比 ρ。本书实现里，$\rho < 0.25$ 就把半径缩为四分之一；$\rho > 0.75$ 且步子靠近圈边时扩大半径；$\rho > 0.1$ 才接受新位置。这些数是具体实现选择，不是数学唯一规定。

白话说：地图预测总不准，就少走点；一直顶到圈边、预测又很准，就可以适当放宽。若一个很小的圈内步都预测不准，光把圈画大通常不能解决问题。

下面在同一个小问题上比较 GN、LM、狗腿。先看三者是否接近 $(1,1)$，再观察各自总分怎样

变化。它们都能在这个例子成功，不意味着每个问题都应选同一个方法。

```
def fun(x):
    return np.array([10*(x[1]-x[0]**2),1-x[0]])
def jac(x):
    return np.array([[-20*x[0],10.],[-1.,0.]])
results = {
    'GN':gauss_newton(fun,jac,[-1.2,1.]),
    'LM':levenberg_marquardt(fun,jac,[-1.2,1.]),
    'dogleg':dogleg(fun,jac,[-1.2,1.])
}
for name,result in results.items():
    print(name,result.x,result.status,'attempts:',len(result.history)-1)
    plt.semilogy([max(p['cost'],1e-20) for p in result.history],'.-',label=name)
    assert np.allclose(result.x,[1,1],atol=1e-5)
plt.xlabel('attempt (different per-method cost)');plt.ylabel('cost')
plt.legend();plt.show()
```

```
GN [1. 1.] gradient attempts: 10
LM [1. 1.] gradient attempts: 17
dogleg [1. 1.] gradient attempts: 14
```

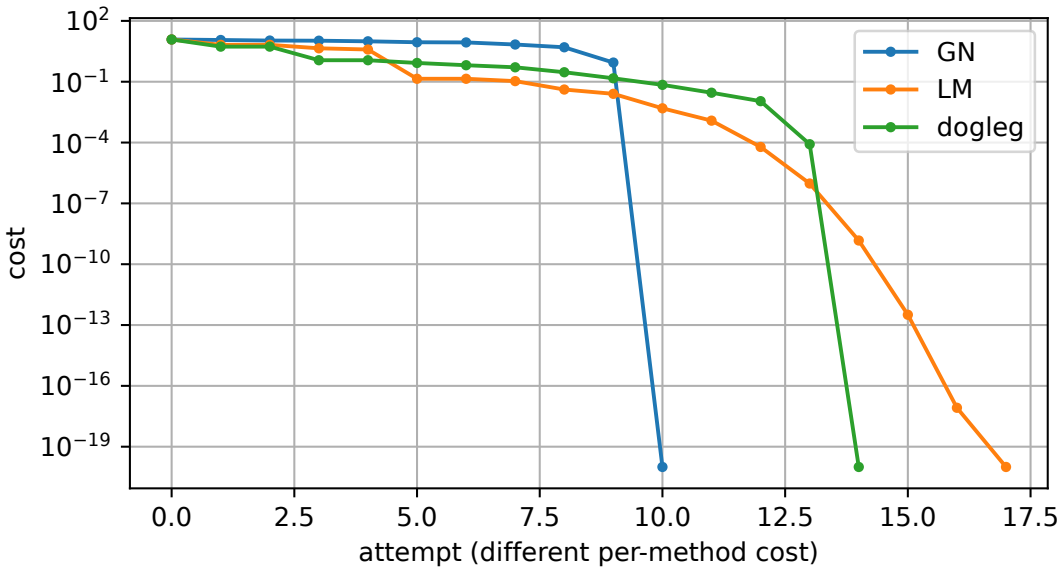


图 11.2: 三种局部方法在相同问题上的目标历史

11.4 不要从一张曲线宣布速度冠军

横轴 `attempt` 是尝试次数，不是秒数。不同方法每次需要的残差计算、雅可比计算和矩阵运算不同。比较效率要固定数据、起点、目标和精度，再统计实际求值次数与耗时。

11.5 动手前先确认你在改哪个实验

第一个画圈实验的 `radius = 1.` 特意选在第三种分支。**不要只把它随意改成极大或极小，再照用这一格的交点公式**：此时可能应走第一或第二分支，原断言也不再适用。

若想研究完整算法的小半径起步，可查看 `lsq/solvers.py` 中 `dogleg` 的 `initial_radius` 参数，在下方完整求解器调用中修改初始半径，再比较历史。先用 `dogleg?` 或查看函数定义确认接口。小半径初期通常更谨慎，但总次数仍与问题有关。

问题：SciPy 的 `dogbox` 就是本章球形狗腿法吗？

解答：不是。它采用矩形信赖域并处理边界，名字相似不能当作同一个实现。

进阶核对：原参考书部分狗腿记法存在已记录疑点，本书始终从 $q(0)-q(h)=-g^{T}h-\frac{1}{2}\|Jh\|^{2}$ 计算预测下降。这个来源比记住某个特殊分支公式更稳妥，详见附录 D。

第十二章　读数不完美时：单位、边界和异常点

前面的读数只是轻轻抖动。实际工作中，可能还混进一条通信错误的记录，某个参数不能为负，几个参数的数值大小又差了几千倍。换一个求解器名称，不能同时解决所有问题。

本章先用**人为加入异常点的合成数据**练手，学习面对真实数据时要做哪些检查。我们没有在这里采集真实仪器数据。

12.1　先把旋钮的刻度说清楚

调时间常数 1 秒和调长度 1 毫米，不一定是同样大的改变。**参数缩放**是给每个参数一个合理的参考刻度，使“调一格”表达相近的实际变化程度。代码的 `x_scale=[2.,1.,.5]` 就提供这样的尺度。

公式上令 $x = Sz$，其中 S 的对角线放各参数尺度，于是 $J_z = J_x S$。白化处理的是不同读数的误差尺度，参数缩放处理的是不同旋钮的刻度；一个主要作用于行，一个主要作用于列，不能互相替代。

12.2　有些位置根本不允许走

下降速率若按模型要求非负，就明确设置下界 0。这叫**边界约束**。本例用 SciPy 的 `trf`，它支持上下界。不能先算任意一步、最后随意裁剪，再认为仍是同一种优化过程：裁剪可能改变方向和原先的改善预测。SciPy 的 `lm` 不支持边界，且只支持普通平方损失。

12.3　不让几个离谱读数牵着整条线走

稳健损失是一种评分规则：小偏差仍认真计分，特别大的偏差不再按平方那么快地涨分。像开会时仍听取离群意见，但不让一句特别响的话压过所有证据。

下面在第 5、18、30 号数组位置人为加入大偏差，比较普通平方评分与 `soft_l1`。图中 `contaminated data` 是混入异常值的读数，`squared loss` 是普通拟合，`soft-L1` 是稳健拟合。先猜哪条曲线更少追着那几个孤立点跑。

需要查公式时：标准化平方残差为 s 时，soft-L1 用 $\rho(s) = 2(\sqrt{1+s} - 1)$。尺度 `f_scale` 决定多大的原始残差开始明显降低影响，本例设为正常噪声 `.04`。这个尺度应有测量依据，不能只为画出喜欢的线而挑选。

```
rng = np.random.default_rng(112)
```

```
t = np.linspace(0,4,60); sigma=.04
truth = np.array([2.5,.8,.4])
y = truth[0]*np.exp(-truth[1]*t)+truth[2]+rng.normal(0,sigma,len(t))
y[[5,18,30]] += [1.2,-1.1,1.3]
def residual(p):
    return p[0]*np.exp(-p[1]*t)+p[2]-y
bounds = ([0.,0.,0.],[6.,4.,2.])
ordinary = least_squares(residual,[2.,.5,.2],bounds=bounds,method='trf')
robust = least_squares(residual,[2.,.5,.2],bounds=bounds,
                       method='trf',loss='soft_l1',f_scale=sigma,
                       x_scale=[2.,1.,.5])
print('truth:',truth,'ordinary:',ordinary.x,'robust:',robust.x)
plt.scatter(t,y,s=12,label='contaminated data')
plt.plot(t,residual(ordinary.x)+y,label='squared loss')
plt.plot(t,residual(robust.x)+y,label='soft-L1')
plt.xlabel('time');plt.ylabel('response');plt.legend();plt.show()
assert np.all(robust.x >= np.array(bounds[0]))
assert np.all(robust.x <= np.array(bounds[1]))
```

```
truth: [2.5 0.8 0.4] ordinary: [2.57621 0.85852 0.44442] robust: [2.48872 0.7942  0.39657]
```

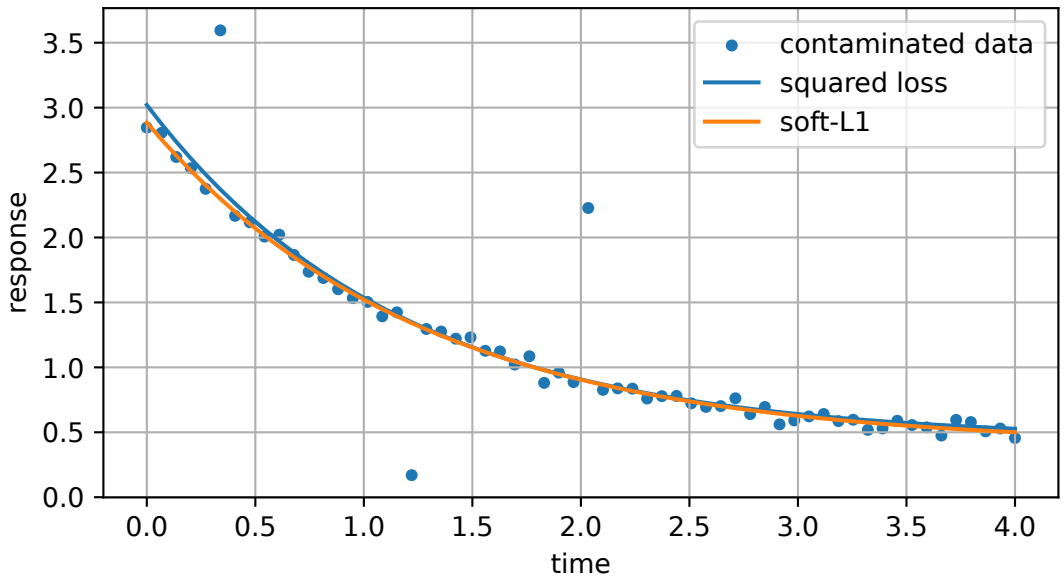


图 12.1: 普通平方与 soft-L1 损失的拟合差别

12.4 降低影响以后，仍然要调查异常点

设备饱和、通信错误和模型遗漏，都可能表现为“大残差”，处理方法却不同。稳健拟合不是删除责任，也不会把错误模型自动修好。要记录哪些数据被降权，以及为什么。

下面换几个起点再拟合同一组数据。`raw RMSE` 是原始残差的均方根：先平方、求平均、再开方，因此回到原读数单位；`robust cost` 是稳健评分。它们回答的问题不同。

特别注意：普通平方和与稳健 cost 的规则不同，**不能直接比较两个 cost 的数值大小，就宣布哪种方法更好**。使用一致的残差指标、独立数据和实际目标来评估。

12.5 结果不对时，按这个顺序排查

先看输入有没有空值、无穷和单位错误，初值有没有越界；再用小扰动核对导数；然后检查参数尺度和哪些组合难以分清；最后画残差随时间、输入和预测值的图。几个起点得出不同答案，是需要调查的信号；几个起点碰巧一致，也不能证明已找遍所有可能答案。

```
starts = [[1.,.1,.1],[4.,2.,1.],[2.5,.8,.4]]
for start in starts:
    f = least_squares(residual,start,bounds=bounds,
                      loss='soft_l1',f_scale=sigma,x_scale=[2.,1.,.5])
    raw_rmse = np.sqrt(np.mean(residual(f.x)**2))
    print('start:',start,'x:',f.x,'robust cost:',f.cost,'raw RMSE:',raw_rmse)
```

```
start: [1.0, 0.1, 0.1] x: [2.48872 0.7942  0.39657] robust cost: 0.1796936775748545 raw RMSE:
     0.28659395362999923
start: [4.0, 2.0, 1.0] x: [2.48872 0.7942  0.39657] robust cost: 0.1796936775748545 raw RMSE:
     0.28659395362432843
start: [2.5, 0.8, 0.4] x: [2.48872 0.7942  0.39657] robust cost: 0.1796936775748545 raw RMSE: 0.2865939536366646
```

12.6 动手试：把正常波动与异常程度分开

先保持原数据不变，只把 f_scale=sigma 试着改为 10*sigma，比较曲线。再恢复尺度，改变加入异常偏差的三个数。一次只改一类，才知道区别来自哪里。

问题一：若残差函数已经除以 sigma，原来设为 sigma 的 f_scale 应改成什么？

解答：改为 1，才保留同样的原始误差转折点。否则同一个测量偏差会受到不同对待。

问题二：稳健拟合后，可以直接用第 05 章的普通最小二乘协方差公式吗？

解答：通常不能。这里权重会随残差变化，还可能碰到边界，需要匹配的方法，例如合理的重采样或稳健协方差估计，并说明假设。

问题三：误差不再下降，怎么知道是仪器噪声的底线，还是算法卡住？

解答：结合重复测量的波动、导数、参数尺度、停止理由和不同起点来查。一个 “success” 或一条水平线都不够。能把原因分清，才知道下一步该换测量、改模型，还是改走法。

第十三章　递推最小二乘

传感器每秒发来一个新读数。如果每次都把从第一秒开始的所有记录重新拟合一遍，运行久了会越来越费事。能不能保留一份有用的摘要，来一条就更新一次？

递推最小二乘（RLS）就是这种“边收数据边修正”的方法。它保留当前参数，以及一张描述各参数方向信息强弱的表。不是只看最后一条；过去的数据已经通过摘要影响了当前结果。

先看下面的直线实验。每收到一条位置读数，就更新起点和速度。图中 `intercept` 是起点估计，`slope` 是速度估计，虚线是真实设置。早期读数少、时间范围窄，两者未必分得清；数据逐渐丰富后才比较稳定。

13.1　把一次更新拆成三句话

1. 用旧参数预测新读数。
2. 算“新读数减预测”的差。
3. 根据当前信息和新观测的作用，决定每个参数修正多少。

第三步的系数叫**增益** K_t，它不是随便设一个固定百分比，而是按计算表和已有信息算出来。

13.2　想知道公式怎么写

模型是 $y_t = \phi_t^T x + \varepsilon_t$。直线例子里 $\phi_t = [1, t]^T$，把截距和斜率变成这一刻的预测。递推式为

$$
\begin{aligned}
K_t &= \frac{P_{t-1}\phi_t}{\lambda + \phi_t^T P_{t-1}\phi_t}, \\
\hat{x}_t &= \hat{x}_{t-1} + K_t(y_t - \phi_t^T \hat{x}_{t-1}), \\
P_t &= \frac{1}{\lambda}\left(P_{t-1} - K_t \phi_t^T P_{t-1}\right).
\end{aligned}
$$

P 是**逆信息矩阵**；在本章给定的尺度下，它记录哪些参数组合约束强、哪些仍弱。只有结合噪声尺度和模型，才能进一步把它解释成参数协方差，不能看到字母 P 就直接称作置信度。

这些式子对应的批量目标是

$$
S_t(x) = \frac{1}{2}\sum_{i=1}^{t} \lambda^{t-i}(y_i - \phi_i^T x)^2 + \frac{\lambda^t}{2}(x - x_0)^T P_0^{-1}(x - x_0).
$$

第一项累积读数误差，第二项保留初始参数预期；λ 在下一节解释。P_0 大表示初始预期较弱，但有限值并非完全没有预期。因此代码和批量比较时保留了同一个小惩罚。

```
rng = np.random.default_rng(113)
t = np.linspace(0,5,80); A = np.column_stack([np.ones_like(t),t])
y = A@np.array([1.,2.])+rng.normal(0,.2,len(t))
beta = np.zeros(2); P = 100*np.eye(2); track=[]
for phi, yi in zip(A,y):
    beta,P = recursive_update(beta,P,phi,yi)
    track.append(beta.copy())
batch = np.linalg.solve(A.T@A+.01*np.eye(2),A.T@y)
print('RLS:',beta,'regularized batch:',batch)
assert np.allclose(beta,batch,atol=1e-10)
track=np.array(track)
plt.plot(track[:,0],label='intercept');plt.plot(track[:,1],label='slope')
plt.axhline(1,color='gray',linestyle='--');plt.axhline(2,color='gray',linestyle='--')
plt.xlabel('sample count');plt.ylabel('estimate');plt.legend();plt.show()
```

```
RLS: [1.01505 2.00242] regularized batch: [1.01505 2.00242]
```

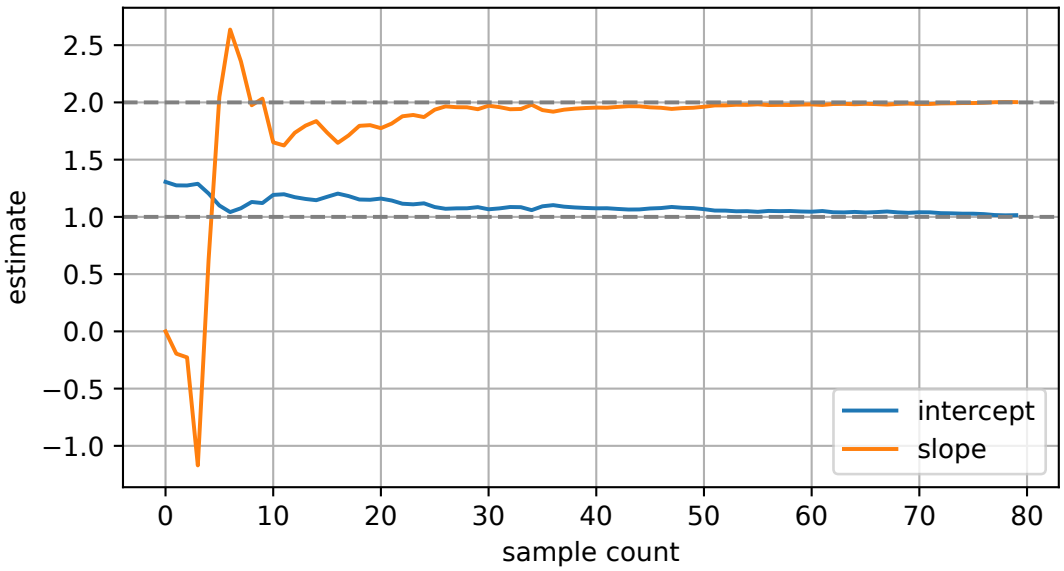


图 13.1: RLS 参数随样本到来的变化

13.3 如果对象真的变了，就不能永远平均旧答案

下面的真实水平先是 1，后来变成 2。**遗忘因子** λ 决定旧证据的折扣：等于 1 时保留全部，略小于 1 时让旧信息慢慢减轻。它也会让初始先验的影响一起衰减。

先看曲线在跳变后追得多快，再看稳定阶段抖得多大。`lambda=0.97` 通常更容易跟上变化，也更容易受新噪声影响。**适应快和稳定并不是免费同时得到的。**

常说有效记忆长度约为 $1/(1-\lambda)$，例如 .97 大约对应 33 条记录的权重总量。这不是第 34 条

以前统统删除。100 步前的权重仍是 $.97^{100} \approx 0.0476$，只是越来越轻。

```
signal = np.r_[np.ones(80),np.full(80,2.)]
measurements = signal+rng.normal(0,.15,len(signal))
for lam in (1.,.97):
    b=np.zeros(1);p=100*np.eye(1);estimates=[]
    for yi in measurements:
        b,p=recursive_update(b,p,[1.],yi,forgetting=lam)
        estimates.append(b[0])
    plt.plot(estimates,label=f'lambda={lam}')
plt.plot(signal,'k--',label='changing truth')
plt.xlabel('time index');plt.ylabel('level');plt.legend();plt.show()
```

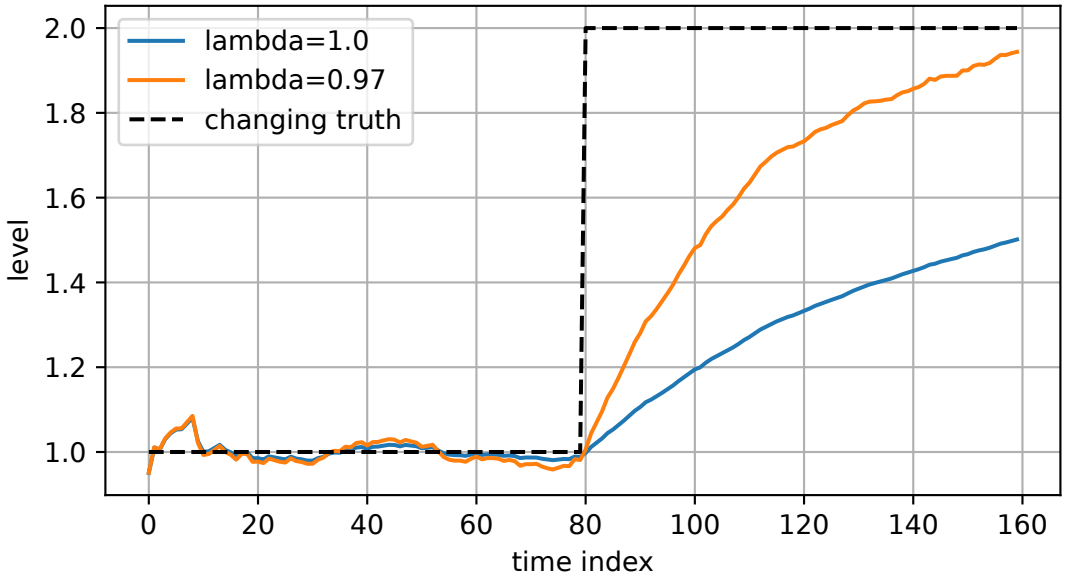


图 13.2: 遗忘因子对变化水平的跟踪

13.4 它和卡尔曼滤波为什么像，又为什么不能随意画等号

两者都有“先预测，再用新证据修正”的结构。在线性观测、常参数、相应先验和噪声假设下，RLS 可以看成估计静态参数的一种卡尔曼方法。

但如果对象本身会动，卡尔曼模型还可以明确写出它怎样变化，以及变化有多不确定，叫**动力学**和**过程噪声**。遗忘因子主要是给旧证据统一打折，含义不同；不能忽略条件就说所有 RLS 都等于同一个卡尔曼滤波器。

13.5 公式写得相等，电脑算起来未必同样稳定

上面的 P 更新含有两个相近数相减。初始 P 很大时，有限位小数可能把有用差异消掉。教学代码改用等价的 **Joseph 形式**：令 $M = I - K_t\phi_t^T$，计算

$$P_t = M(P_{t-1}/\lambda)M^T + K_t K_t^T.$$

再把因舍入产生的细小不对称调回对称。这个式子对应本章单位观测噪声的信息尺度，多做一些乘法换取可靠性；长期高精度系统还应研究平方根或 QR 形式。能跑通小例子，不是工业长期运行保证。

13.6 动手试与解答

把第二个实验里的 `.97` 改为 `.9`，先猜跳变后追得更快还是更慢，稳定后更平还是更抖，再运行比较。

问题：为什么第一个实验的批量对照有 `.01*np.eye(2)`？

解答：初始 $P_0 = 100I$，其逆是 $0.01I$，相当于保留一份很弱的零参数预期。去掉它，就不是在比较同一个目标。

问题：参数其实不变、噪声独立时，是不是遗忘越快越好？

解答：不是。旧读数仍有用，过快遗忘会舍弃有效信息，使估计更抖。先说明对象为何会变，再决定是否付出这份代价。

第十四章　综合案例：曲线拟合与标定

现在来做一个完整任务：某个传感器启动后，读数先高后低，逐渐稳定。你想估计“起初高出背景多少、下降多快、背景在哪里”，以后才能根据这个响应使用或校准它。

标定就是通过一组有规则的测量，找出装置的响应关系；不是只让电脑吐出三个小数。我们还要知道答案在新读数上表现怎样，有哪些地方仍拿不准。

14.1　拟合前，先写清实验约定

本例使用 `data/decay-calibration.csv`，是明确标注的合成数据，并非真实传感器。`time_s` 是秒，`response` 是响应，`noise_sigma` 是每条记录已知的噪声标准差；响应使用一套一致的任意单位。

每五行先留出第一行作检查，其余用于拟合。**留出数据**像暂时封起来的试题，不参与选初值和参数。这里按这个方式演示流程；真实时间序列有相关性或漂移时，如何留出要按预测任务重新设计，不能照抄行号规则。

模型仍是 $f(t) = ae^{-kt} + c$，用“预测减读数，再除该次噪声尺度”评分。三个参数在本例都要求非负；这是本例的物理假定，不是所有传感器背景都必须大于零。

14.2　起点从数据来，不偷看模拟答案

先列一些候选下降速度 k。固定一个 k 后，幅值 a 和背景 c 可以一次线性求解。挑训练误差小的一组作初值，再做带边界的非线性优化。

下面看 `data-derived initial` 与 `fit`，再看训练和留出数据的 `normalized RMSE`。这是标准化残差的均方根，约 1 表示误差与假定噪声大致同量级，但有限样本、拟合和模型误差都影响它；不能把接近 1 当成自动验收印章。

```
data=np.genfromtxt(root/'data/decay-calibration.csv',delimiter=',',names=True)
t,y,sigma=data['time_s'],data['response'],data['noise_sigma']
train=np.arange(len(t))%5!=0
tt,yy,ss=t[train],y[train],sigma[train]
candidates=[]
for k in np.linspace(.1,1.5,50):
    A=np.column_stack([np.exp(-k*tt),np.ones_like(tt)])
    ac=np.linalg.lstsq(A/ss[:,None],yy/ss,rcond=None)[0]
    if ac[0]>=0:
        cost=np.sum(((A@ac-yy)/ss)**2)
```

```
        candidates.append((cost,np.array([ac[0],k,ac[1]])))
initial=min(candidates,key=lambda z:z[0])[1]
initial[2]=max(0.,initial[2])
def model(p,t):
    return p[0]*np.exp(-p[1]*t)+p[2]
def residual(p):
    return (model(p,tt)-yy)/ss
def jac(p):
    e=np.exp(-p[1]*tt)
    return np.column_stack([e,-p[0]*tt*e,np.ones_like(tt)])/ss[:,None]
fit=least_squares(residual,initial,jac=jac,bounds=(0,np.inf),x_scale=[3,1,.3])
print('data-derived initial:',initial,'fit:',fit.x,'status:',fit.message)
print('training normalized RMSE:',np.sqrt(np.mean(residual(fit.x)**2)))
heldout=(model(fit.x,t[~train])-y[~train])/sigma[~train]
print('held-out normalized RMSE:',np.sqrt(np.mean(heldout**2)))
assert fit.success and np.all(np.isfinite(fit.x))
```

```
data-derived initial: [2.78596 0.67143 0.2823 ] fit: [2.79075 0.66143 0.26947] status: `ftol` termination
     condition is satisfied.
training normalized RMSE: 0.8496890299942883
held-out normalized RMSE: 1.07904696484996
```

14.3 曲线好看，三个参数就都测准了吗

不一定。早期幅值高一点、背景低一点，可能仍画出相近曲线。先看白化雅可比的奇异值，检查有无难以分清的参数组合。它是局部诊断，不是所有参数可能性的完整地图。

活动边界是参数已经顶到允许的上限或下限。例如背景被卡在 0，此时它的误差不能想当然画成左右对称。代码先检查是否数值满列秩、有没有活动边界；不满足就打印 `uncertainty omitted`，明确不报这类区间。

这里噪声尺度已知，条件满足时使用局部近似

$$C_x \approx (J^T J)^{-1}.$$

C_x 是参数协方差，表达重复测量时参数估计的共同波动；不用再乘一次训练残差方差。若信息不足，伪逆虽然能返回一个矩阵，也不能把不清楚的方向解释成“方差为零”。曲线强烈非线性时，这个局部正态近似本身也可能不准。

14.4 图上的带子，到底包住什么

左图 `train` 是用于拟合的点，`held-out` 是留下检查的点。`mean 95% approx.` 阴影描述**平均响应曲线**的不确定性，不是要求 95% 的单次新读数落进去。新读数还带仪器噪声，预测它需要更宽、且按相应方法构造的预测区间。

公式上，在新时间 t，模型均值的一阶方差是 $j(t)C_x j(t)^T$；预测一次未来观测，还要在适当独立假设下加上观测噪声方差。右图是标准化残差，先看是否围绕零散开，再找有没有随时间变化的弯曲或偏向。

```
from lsq import linearized_covariance
J=jac(fit.x)
s=np.linalg.svd(J,compute_uv=False)
print('Jacobian singular values:',s)
try:
    cov=linearized_covariance(J,fit.active_mask)
    print('approximate parameter standard errors:',np.sqrt(np.diag(cov)))
except ValueError as error:
    cov=None
    print('uncertainty omitted:',error)
grid=np.linspace(0,5,160);e=np.exp(-fit.x[1]*grid)
Jmean=np.column_stack([e,-fit.x[0]*grid*e,np.ones_like(grid)])
prediction=model(fit.x,grid)
fig,ax=plt.subplots(1,2,figsize=(7,3))
ax[0].scatter(tt,yy,s=10,label='train')
ax[0].scatter(t[~train],y[~train],marker='x',label='held-out')
ax[0].plot(grid,prediction)
if cov is not None:
    se=np.sqrt(np.maximum(0,np.einsum('ij,jk,ik->i',Jmean,cov,Jmean)))
    ax[0].fill_between(grid,prediction-1.96*se,prediction+1.96*se,
                       alpha=.25,label='mean 95% approx.')
ax[0].set(xlabel='time (s)',ylabel='response');ax[0].legend(fontsize=7)
ax[1].scatter(tt,residual(fit.x),s=12)
ax[1].axhline(0,color='gray');ax[1].set(xlabel='time (s)',ylabel='normalized residual')
plt.tight_layout();plt.show()
```

```
Jacobian singular values: [243.55618  57.36661  29.09522]
approximate parameter standard errors: [0.02382 0.01723 0.02525]
```

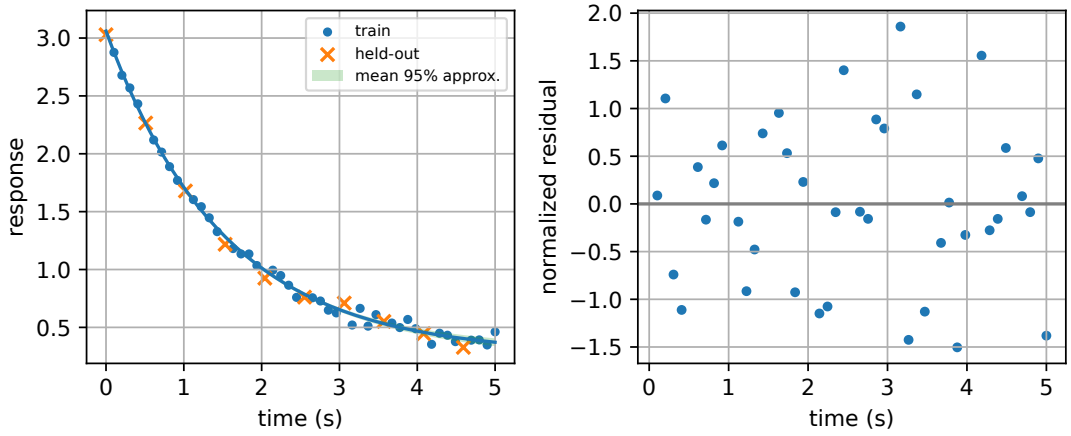


图 14.1: 完整标定结果、均值置信带与标准化残差

14.5 把结果写成别人能复查的话

不要只写“拟合成功，参数为……”。至少说明：使用哪个模型和单位，哪些数据拟合、哪些检查，噪声尺度从哪来，初值和边界是什么，程序为什么停止，训练与留出残差怎样，以及区间依赖什么假设。

可以照着填：**“在这些测量条件下，用……模型描述响应；以……作为噪声尺度，得到……参数；留出读数的误差为……；残差图呈现……；这些区间只在……条件下适用。”** 实测之前不要把模拟数据的结论改写成设备性能保证。

14.6 自己试与核对

先遮住参数结果，只看输入数据和图，猜下降快慢及背景大概位置。再运行。最后试着只保留很早的观测，看看参数是否更难分清。改数据后需同步调整训练选择和画图，别把原例的单次测试当作普遍断言。

为什么早期难分清？当 kt 很小时，$ae^{-kt}+c \approx (a+c)-akt$。读数主要看见“幅值与背景之和”和“幅值乘速率”，不像完整曲线那样能分别提供信息。多量一些较晚时间点通常更有帮助。

问题：反复用留出数据选模型后，还能叫最终独立检查吗？

解答：不能，它已经参与决策，需要新的测试数据或合适的嵌套验证。

问题：残差有明显弯曲，把误差条画宽一点能解决吗？

解答：不能。先查模型遗漏、时间响应或噪声假设。区间公式不会自动修复系统性偏差。即使模拟真模型已知，一次实验的 95% 区间也不承诺必定覆盖真参数。

第十五章　综合案例：PWM→XYZ 校正

想象你有红、绿、蓝三路灯。你想让测量仪看到某个指定的颜色和亮度。把红灯调亮，不一定只改变一项读数，因为三路光会混在一起。我们需要边测边调，而不是把三个误差各自交给一盏灯处理。

这是本书最后一个综合实验：**从想要的输出倒推应该怎样设置输入**，叫逆问题。本例只是仿真练习，不能直接当成真实屏幕的校正方案。

15.1　先翻译名字，再谈算法

- **PWM**：脉冲宽度调制，可以粗略理解成快速开关的“开灯时间比例”。本实验用 0—4095 的整数指令代表三路控制量，值越大通常越亮，但不是说指令翻倍、发光量就必定翻倍。
- **XYZ**：用三个数记录颜色测量结果的坐标系统，不等于红、绿、蓝三个灯的亮度。标准颜色测量中 Y 与亮度相关，本例用归一化数值作教学仿真。
- **通道耦合**：动一个旋钮，多个输出一起变。这里用一张固定混色表表示。
- **量化**：控制值只能落在整数格子上，想走 0.2 格，设备可能根本没变。

15.2　本实验故意简化了哪些地方

指令低于 96 时视为关闭；96 以上，各通道响应连续、分段线性且单调。也就是每一小段用直线连接，往上调不会让该通道自身变暗。固定的正混色矩阵把发光量换成 XYZ；每次测量叠加标准差 2×10^{-4} 的独立高斯噪声。

真实设备可能有发热、限流、通道互相影响等现象，必须重新测量。下面图的 `channel response` 是各通道响应，`12-bit PWM` 是控制指令；先找低端的平台和各段斜率，不要把模拟折线误认成设备实测曲线。

目标评分仍可写成 $\frac{1}{2}\|M(p)-X_{\text{target}}\|^2$，$p$ 是三路指令，M 是测量函数。XYZ 波动不同就应按噪声处理；本例三项取相同噪声尺度，而且这里的欧氏 XYZ 误差不等同于人眼感知色差。

```
mix=np.array([[.52,.29,.18],[.23,.70,.07],[.03,.12,.94]])
knots=np.array([0,96,512,2048,4095.])
levels=np.array([[0,0,.045,.40,1.],
                 [0,0,.055,.44,1.],
                 [0,0,.035,.37,1.]])
def simulator(p):
    p=np.clip(np.rint(p),0,4095)
```

```
    light=np.array([np.interp(p[i],knots,levels[i]) for i in range(3)])
    return mix@light
rng=np.random.default_rng(115)
def measure(p,repeats=8):
    return simulator(p)+rng.normal(0,2e-4,(repeats,3)).mean(axis=0)
def measured_jacobian(p,delta=64):
    J=np.zeros((3,3))
    for k in range(3):
        plus=p.copy();minus=p.copy()
        plus[k]=min(4095,p[k]+delta)
        minus[k]=max(96,p[k]-delta)
        span=plus[k]-minus[k]
        J[:,k]=(measure(plus)-measure(minus))/span
    return J
target=simulator([820,960,740])
start=np.array([1100.,900.,1000.])
print('target XYZ:',target,'initial error:',np.linalg.norm(simulator(start)-target))
grid=np.linspace(0,4095,200)
for k in range(3):
    plt.plot(grid,np.interp(grid,knots,levels[k]),label='RGB'[k])
plt.xlabel('12-bit PWM');plt.ylabel('channel response');plt.legend();plt.show()
```

```
target XYZ: [0.12418 0.14976 0.1032 ] initial error: 0.06697135746402544
```

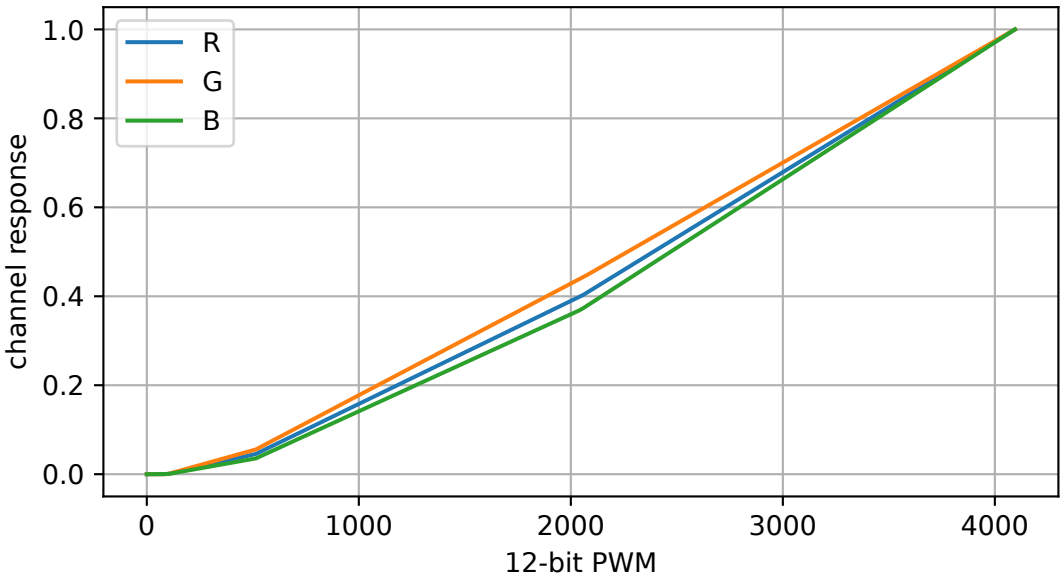


图 15.1: PWM 通道的合成分段单调响应

15.3 先轻轻试旋钮，量出这附近的响应

每次只改一个通道，另外两个固定；向上、向下各试一点，计算 XYZ 差，再除实际指令差。这就是第 07 章影响表的实测版本。

边界处尤其要小心：想减 64，但已经快到底，就只能减更少。**分母要用实际走过的跨度，不能仍写成 128**。扰动太小会被噪声和整数格子淹没，太大又可能跨过好几种响应斜率。

代码默认每次平均 8 个读数，减轻独立随机波动；如果仪器一直同方向漂移，平均不会自动消掉它。把参数先写成 $u = p/4095$，则 0—1 代表整个控制范围，让计算尺度更合适。

15.4 把前面几章连起来

下面每轮先用阻尼求一个局部步，再把每个通道单次改动限制在 128 以内，四舍五入到整数后实际测量。误差变小才接受；没改善就加大阻尼。它还保留已测过的最好位置，避免尝试失败后把更差设置当成果。

先看打印的 `status`，再看 `noise-free error`。后者借助模拟器知道无噪声输出，仅用于本教学检查；真实仪器通常没有这个可偷看的真值。

图中 `measured XYZ error` 是测得误差大小，灰色虚线是本次容差。**容差**是允许差多少，和“程序还能否继续降低误差”不是同一个条件。

```
def calibrate(target,start,max_iter=45,tolerance=1.2e-3):
    target=np.asarray(target,dtype=float)
    start=np.asarray(start,dtype=float)
    if (target.shape!=(3,) or start.shape!=(3,) or
            not np.all(np.isfinite(target)) or not np.all(np.isfinite(start))):
        raise ValueError('target and start must be finite three-vectors')
    if (not isinstance(max_iter,(int,np.integer)) or max_iter<1 or
            np.ndim(tolerance)!=0 or not np.isfinite(tolerance) or tolerance<=0):
        raise ValueError('positive finite tolerance and integer budget required')
    p=np.clip(np.rint(start),96,4095).astype(float)
    lower=simulator([96]*3);upper=simulator([4095]*3)
    box_distance=np.linalg.norm(np.clip(target,lower,upper)-target)
    if box_distance>tolerance:
        return p,'unreachable_box',[]
    current=measure(p);error=np.linalg.norm(current-target)
    best=p.copy();best_error=error
    mu=1e-4;history=[error];rejects=0
    for _ in range(max_iter):
        if error <= tolerance:
            return p,'target_reached',history
        J=measured_jacobian(p)*4095
        residual=current-target
        du=np.linalg.lstsq(np.vstack([J,np.sqrt(mu)*np.eye(3)]),
                           np.r_[-residual,np.zeros(3)],rcond=None)[0]
        step=4095*du
        step*=min(1.,128/max(np.max(np.abs(step)),1e-12))
        candidate=np.clip(np.rint(p+step),96,4095)
        if np.array_equal(candidate,p):
            return best,'quantized_stall',history
        value=measure(candidate);candidate_error=np.linalg.norm(value-target)
        if candidate_error < error:
```

```
            p,current,error=candidate,value,candidate_error
            rejects=0;mu=max(mu/3,1e-9)
            if error<best_error:
                best,best_error=p.copy(),error
        else:
            rejects+=1;mu*=4
        history.append(error)
        if rejects>=8:
            return best,'no_measured_progress',history
    if error<=tolerance:
        return p,'target_reached',history
    return best,'budget_exhausted',history
p,status,history=calibrate(target,start)
print('PWM:',p,'status:',status,'noise-free error:',np.linalg.norm(simulator(p)-target))
assert np.all((p>=96)&(p<=4095))
assert np.linalg.norm(simulator(p)-target)<.003
plt.semilogy(history,'o-');plt.axhline(1.2e-3,color='gray',linestyle='--')
plt.xlabel('attempt');plt.ylabel('measured XYZ error');plt.show()
_,unreachable,_=calibrate(np.array([2.,2.,2.]),start)
print('unreachable target:',unreachable)
assert unreachable != 'target_reached'
```

```
PWM: [821. 959. 739.] status: target_reached noise-free error: 0.00026654803100767464
unreachable target: unreachable_box
```

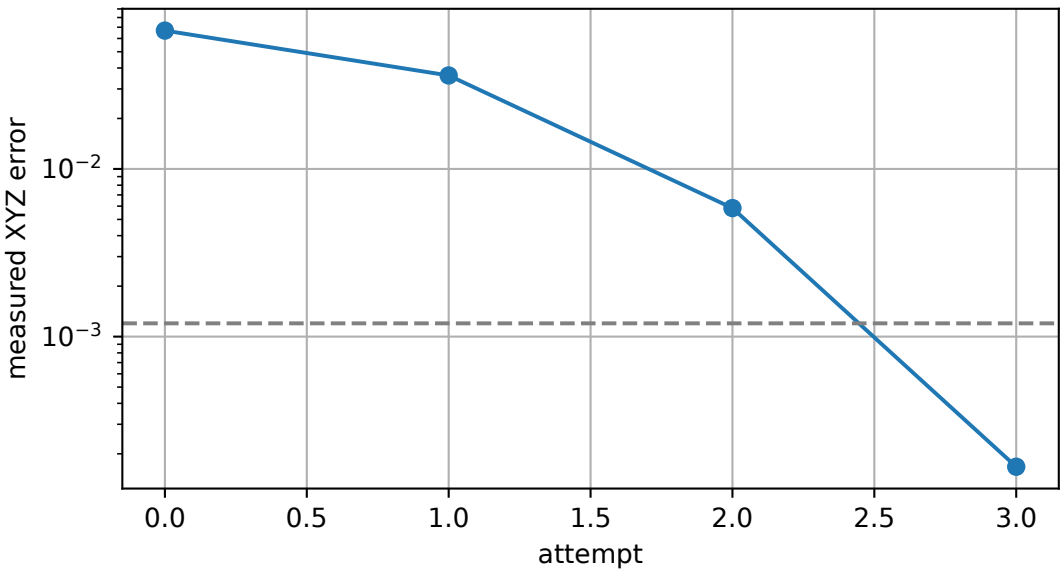


图 15.2: 带噪声 PWM 校正的实测误差历史

15.5 没达到目标时，先说清为什么停

返回状态	白话意思
`target_reached`	当前平均测量误差已在本次容差内
`unreachable_box`	目标离最粗的允许范围都太远，提前排除
`quantized_stall`	建议步取整后没挪动，卡在格子上
`no_measured_progress`	连续尝试仍没测到改善
`budget_exhausted`	允许尝试次数用完，仍未达标

低灰平台上，拧一点也不变，雅可比可能接近零；这不是已经达到目标的证明。需要更有效的扰动、跨平台探测，或明确报告无法分辨。本例从 96 以上开始，没有解决所有真实低灰问题。

包围盒只是分别给 X、Y、Z 各画最大最小范围。三项单看都不超范围，组合起来仍可能达不到，好比身高体重各在允许区间，却不代表任意搭配都存在。设备真正能产生的颜色集合叫**色域**。这里的包围盒只能排除一部分不可能目标，不能证明剩下的都可达。

15.6 达标以后，也要重新验一次

本例按带噪声的测量选最好点，可能挑中了“恰好读得好看”的一次，这叫噪声选择偏差。真实验收应固定最终设置，再独立重复测量，按规定的颜色和亮度容差检查，并记录热稳定时间、顺序和环境。

15.7 动手试与解答

把 `measured_jacobian` 的 `delta=64` 改为 `1`，从头运行比较。更小不一定更精细：响应差变小，噪声经相除被放大，量化也更明显。若原来的验收断言不再通过，这是实验揭示了方法条件，别删掉断言假装仍达标。

问题一：为什么不能分别用 X、Y、Z 的误差只调 R、G、B?

解答：每路灯同时影响多个测量分量，必须考虑一起变化的关系。

问题二：增加蓝通道，测得 Y 反而下降，本仿真能解释吗?

解答：不能。本模型的混色系数和通道斜率均非负。应排查真实耦合、限流、热状态、测量时序和图样条件，先改正对设备的认识，再挑算法。

能清楚报告“设置是多少、误差多少、为什么停、哪些假定还没验证”，比只给一个所谓最优 PWM 更接近可用的工程结果。

附录 A　必要数学基础

这一页是工具箱，不是入场考试。正文遇到陌生符号，再回来找对应工具。第一次先读每节第一段，理解它在记录什么，再看公式。

A.1　一排数、一张表：向量与矩阵

向量就是按顺序放的一排数，例如 `[起点，速度]`。写作 $x \in \mathbb{R}^n$ 只是说“这排有 n 个实数”。**维度**就是数量或形状，不必想到另外一个世界。

矩阵是一张数表。$A \in \mathbb{R}^{m\times n}$ 是 m 行 n 列：本书里每行一条观测，每列一个参数的作用。Ax 把参数转成各次预测；A^T 是交换行列。矩阵相乘要求前一个的列数等于后一个的行数，像插头插座需要对得上。Python 的一维数组没有明确行列方向，查看 `.shape` 能帮助防止误接。

A.2　点积和范数：比较方向，汇总大小

内积（点积）把对应项相乘再加起来：$x^T y = \sum_j x_j y_j$。例如 $(1,2)$ 与 $(3,4)$ 得到 $1\times3+2\times4=11$。点积为零叫正交；对非零向量可理解成垂直。零向量也与所有向量点积为零，但不能给它定义普通方向夹角。

范数是给一排数算整体大小的规则。$\|x\|_2 = \sqrt{\sum_j x_j^2}$ 就是勾股定理的长度，例如 $(3,4)$ 的长度是 5；$\|x\|_\infty = \max_j |x_j|$ 则取最大绝对值。后者可用来检查每个参数是否都足够小，而不是被其他坐标遮住。

A.3　导数、梯度、雅可比、Hessian：四种变化记录

导数记录一个量稍变时，另一个量变多快；**偏导**先固定其他量，只动其中一个。**梯度**收集“每个参数怎样改变一个总分”；**雅可比**收集“每个参数怎样改变每个残差”；**Hessian** 再记录总分的变化率怎样改变，也就是弯曲情况。

对 $F = \frac{1}{2} r^T r$，把逐层变化连接起来的链式法则给出 $\nabla F = J^T r$。足够光滑时，附近目标可以写成

$$F(x+h) = F(x) + g^T h + \frac{1}{2} h^T H h + O(\|h\|^3).$$

右边依次是原分数、斜率作用、弯曲作用和剩下的小误差。$O(\|h\|^3)$ 是误差阶的简写，需要相应光滑条件；仅二阶连续可微时，更一般地写 $o(\|h\|^2)$，表示它相对步长平方越来越不显眼。

A.4 正定、特征值、奇异值：碗是否朝上，哪些方向看得清

实对称矩阵 B **正定**，指所有非零 x 都满足 $x^TBx > 0$，对应二次形状朝上的碗；**半正定**允许某些方向完全平坦。前者二次目标有唯一最低点，后者未必唯一。J^TJ 总半正定，只有 J 满列秩时才正定。

特征值描述矩阵在某些特殊方向上的缩放；实对称矩阵特征值全正等价于正定。**奇异值**适用于更一般的矩阵，非负，描述不同参数组合传到观测后有多明显；它们的平方是 A^TA 的特征值。

最大与最小非零奇异值之比描述看得见的那些方向之间的敏感性差别。如果还存在完全看不见的方向，整体逆问题通常称条件数无穷大；不能因为只算了非零那部分的有限比值，就说所有参数都能确定。

```
A=np.array([[1.,0.],[1.,1.],[1.,2.]])
singular=np.linalg.svd(A,compute_uv=False)
eigen=np.linalg.eigvalsh(A.T@A)
print('singular values squared:',singular**2)
print('eigenvalues (ascending):',eigen)
assert np.allclose(np.sort(singular**2),eigen)
```

```
singular values squared: [7.16228 0.83772]
eigenvalues (ascending): [0.83772 7.16228]
```

A.5 平均位置与共同波动：期望、方差、协方差

期望是在同样规则下反复试验的长期平均位置，不承诺下一次就落在这个数上。**方差**是离平均位置的偏差平方的平均；**标准差**是方差开平方，回到原单位。

协方差记录两个量是否倾向一起增减。一张协方差矩阵的对角线放各自方差，其他格子放成对的共同变化。对随机向量 x，定义为

$$C = \mathbb{E}[(x - \mathbb{E}x)(x - \mathbb{E}x)^T].$$

$\mathbb{E}$ 表示取期望。如果 $z = Bx$ 是线性换算，波动也一起换算：$\mathrm{Cov}(z) = BCB^T$。比如把米换成毫米，数值放大一千倍，方差会放大一百万倍。

协方差为零叫**不相关**；**独立**则更强，意味着知道一个的结果不能增加对另一个的了解。一般不能把两者混同；联合高斯等附加条件下，零协方差才可推出独立。

A.6 拿正文里的式子检查一次

问题一：J 是 20×3，r 是 20 个数，$J^T r$ 有几个数?

提示：先把 J 的行列交换，再检查乘法能否对接。

解答：3 个，对应三个参数的梯度。

问题二：信息不足时，加 μI 是否能让局部步好算？是否同时测准了所有参数?

解答：$\mu > 0$ 时，$h^T(J^T J + \mu I)h = \|Jh\|^2 + \mu\|h\|^2 > 0$，局部表变正定，有助于计算。但这是给步子加限制，不是新增观测，原来分不清的参数仍可能分不清。

附录 B　符号、公式与算法选择

B.1　先按遇到的困难找工具

不用先背完所有缩写。先说“我的数据和目标是什么”，再选第一件值得试的工具。

你遇到的情况	可以从哪里开始	要同时检查什么
未知数只作为已知零件的系数	`np.linalg.lstsq`，内部常用 SVD	是否有重复信息，实际残差多大
有些读数明显更不可靠	按噪声尺度加权，必要时白化	噪声估计依据，误差会不会一起变化
线追着每个点乱摆	降低复杂度，或明确正则化	新数据是否变好，惩罚是否合理
曲线需要反复调整，初值较合理	带回溯的 GN	影响表是否正确，大步是否失准
局部建议步太激进、没有边界要求	LM	参数尺度、接受和拒绝记录
想明确限制这次能调多远	信赖域、狗腿	真实改善是否兑现局部预测
参数有上下限，或有异常点	SciPy `least_squares(method='trf')`	初值合法，损失和噪声尺度匹配
线性读数不断进来	RLS	对象是否变化，要不要遗忘
仪器量化、平台、实测噪声明显	先设计有效扰动，再做受限局部校正	改动是否真的有响应，目标是否能达到

这是一张入口地图，不是按表选完就保证成功。尤其“数据分不清参数”和“局部步算得不好”，是不同困难。

B.2　符号速查：先说人话，再看维度

符号	白话意思	名称与大小
x, h	当前旋钮设置、本轮改动	参数与步，均为 n 个数
y, r	实际读数、预测减读数	观测与残差，均为 m 个数

符号	白话意思	名称与大小
A	每个设置怎样产生各次预测的表	线性设计矩阵，$m \times n$
J	每个旋钮微调后各次残差怎样变	雅可比，$m \times n$
F, g, H	总分、总分的变化率、变化率怎样变	目标、梯度、Hessian
C	读数各自抖多少、是否一起抖	噪声协方差，$m \times m$
μ, Δ	步长费用、这次允许走的半径	阻尼、信赖域半径
λ, D	对最终参数罚多重、罚什么	正则化强度与惩罚算子

第 13 章的 λ 另指遗忘因子；同一个字母在不同上下文可能换工作，不能只看符号猜含义。

B.3 六句能让结果被复查的话

先说模型和单位；说明残差及导数怎样检查；指出哪些参数组合看得清；记录为什么停止；用没参与拟合的数据检验；区分概率近似与工程容差。逐项回答，比不断换算法更容易找到问题。

B.4 公式速查，第二遍阅读时再用

线性最优条件：$A^T(Ax-y)=0$。非线性梯度：$g=J^Tr$。GN 的局部步：$\min_h \|Jh+r\|^2$。LM 的局部步：$\min_h \|Jh+r\|^2+\mu\|h\|^2$。预测下降：$-g^Th-\frac{1}{2}\|Jh\|^2$。参数缩放 $x=Sz$ 后：$J_z=J_xS$。

B.5 下面这段诊断，先看哪几项

先认数组形状，再看秩和奇异值，最后看有限差分与手写雅可比的差。前两项在问信息够不够，最后一项在查变化率有没有写错。打印正常并不能替代残差图。

```
def diagnose_matrix(A):
    s=np.linalg.svd(A,compute_uv=False)
    rank=np.linalg.matrix_rank(A)
    full_column_rank=rank==A.shape[1]
    condition=(s[0]/s[-1]) if full_column_rank else np.inf
    return {'shape':A.shape,'rank':rank,'full_column_rank':full_column_rank,
            'condition':condition}
print(diagnose_matrix(np.column_stack([np.ones(5),np.arange(5.)])))
print(diagnose_matrix(np.ones((5,2))))
assert not diagnose_matrix(np.ones((5,2)))['full_column_rank']
```

```
{'shape': (5, 2), 'rank': np.int64(2), 'full_column_rank': np.True_, 'condition': np.float64(4.738720018687268)}
{'shape': (5, 2), 'rank': np.int64(1), 'full_column_rank': np.False_, 'condition': inf}
```

B.6 检查题

矩阵的秩、条件数和导数核对都正常，是否说明模型适合数据？

提示：一个计算得很准确的直线，能不能仍然不适合加速运动？

解答：可以不适合。矩阵诊断说明这套局部计算和信息结构怎样，残差图才帮助检查模型遗漏。计算正确、假设合理、预测有效，是要分别确认的几件事。

附录 C　进阶方法的入口

这篇是进阶选读。第一次学会直线拟合、加权、检查残差和使用成熟求解器后，已经能做许多事，不必为了“读完”硬啃这里。

这里再问两个实际问题：有些参数能直接算，为什么跟着一起慢慢试？每次重算影响表很贵，能不能利用上一次走路留下的信息？

C.1　能直接算的先算：可分最小二乘

对 $ae^{-kt}+c$，难的是 k。一旦先选定 k，a,c 就只是两种已知零件的系数，可以线性求解。因此可以外层只试下降速度，内层直接算幅值和背景，少让非线性搜索背两个包袱。这叫**变量投影**或可分最小二乘。

一般模型写成 $\Phi(\theta)c$：θ 是需要试的非线性参数，c 是固定它后能直接线性求的系数。

$$c^*(\theta)=\arg\min_c \|\Phi(\theta)c-y\|^2,\qquad \psi(\theta)=\frac{1}{2}\|\Phi(\theta)c^*(\theta)-y\|^2.$$

外层每改一次 θ，内层都要重新求 c^*，不是固定第一次算出的系数。若内层信息结构随 θ 改变，例如某些列突然重复，外层的平滑性和导数也要重新检查。

下面比较变量投影与联合拟合，先看它们是否给出相近参数和残差，而不是只看搜索参数少了几个。

```
rng=np.random.default_rng(203)
t=np.linspace(0,4,40)
y=2.5*np.exp(-.8*t)+.4+rng.normal(0,.025,len(t))
def linear_part(k):
    return np.column_stack([np.exp(-k*t),np.ones_like(t)])
def projected_residual(theta):
    A=linear_part(theta[0])
    ac=np.linalg.lstsq(A,y,rcond=None)[0]
    return A@ac-y
outer=least_squares(projected_residual,[.5],bounds=([.01],[3.]))
ac=np.linalg.lstsq(linear_part(outer.x[0]),y,rcond=None)[0]
print('variable projection [a,k,c]:',np.r_[ac[0],outer.x[0],ac[1]])
assert outer.success
grid=np.linspace(.1,1.5,100)
plt.plot(grid,[.5*np.sum(projected_residual([k])**2) for k in grid])
plt.axvline(outer.x[0],color='C1')
```

```
plt.xlabel('nonlinear rate k');plt.ylabel('profiled cost');plt.show()
```

```
variable projection [a,k,c]: [2.49392 0.81201 0.4154 ]
```

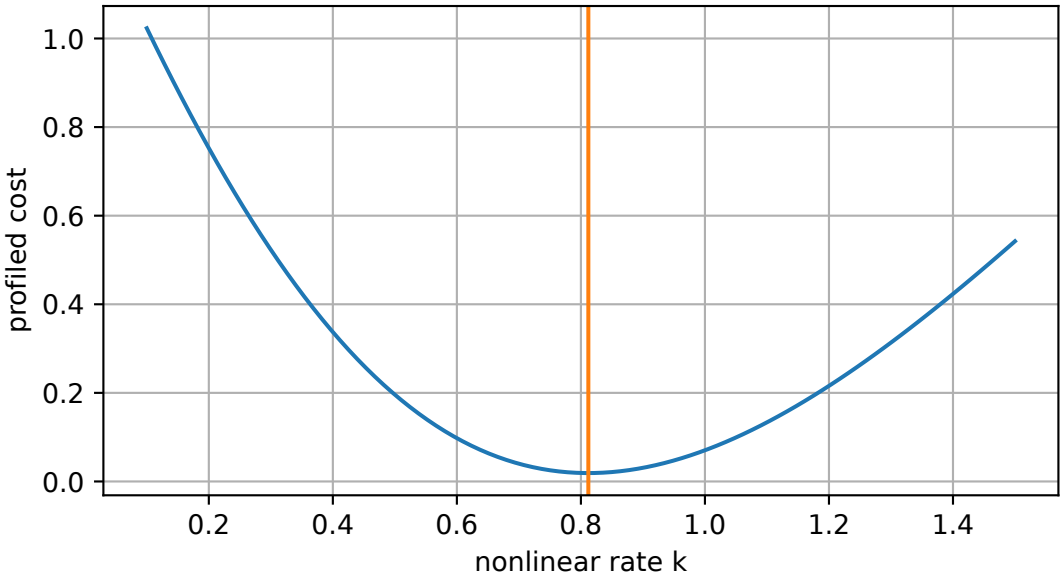


图 C.1: 消去线性参数后的单参数目标

C.2 内层答案也会跟着变

上例让 SciPy 对“每次都重新求过内层的残差函数”做数值微分，因此包含了内层系数随 k 改变的作用。手写外层雅可比时，不能把刚求出的 a, c 当作永远不动；要用相应导数或专门的变量投影公式。

C.3 用走过的一步更新地图：割线条件

假如刚才走了 $s = x_{k+1} - x_k$，残差实际改变了 $y_r = r(x_{k+1}) - r(x_k)$。旧影响表 B 对这步的预测可能不准，我们让新表至少符合刚测到的关系：

$$B_{\text{new}} s = y_r.$$

这叫**割线条件**：利用两个位置的差，校正变化规律。一个常见的秩一更新是

$$B_{\text{new}} = B + \frac{(y_r - Bs)s^T}{s^T s}.$$

“秩一”描述这次修正矩阵的结构，不是说整张表只剩一个独立方向。在把所有格子改变量的平方相加的意义下，也就是 Frobenius 范数，它是满足这一个条件的最小改动。

下面检查新表能否复现这一步实际变化。**符合一个方向的一次观测，不代表所有方向都准确**；噪声、很小的步或过期信息都可能使更新失效，需要重算和保护规则。

```
B=np.array([[1.,.2],[.1,2.],[.4,.3]])
s=np.array([.3,-.2]);delta_r=np.array([.5,-.1,.2])
updated=B+np.outer(delta_r-B@s,s)/(s@s)
print('secant error:',np.linalg.norm(updated@s-delta_r))
assert np.allclose(updated@s,delta_r)
```

```
secant error: 5.551115123125783e-17
```

C.4 为什么还会有混合与拟牛顿方法

GN 没有使用完整曲率。残差较大、被省略的 $\sum_i r_i \nabla^2 r_i$ 重要时，可以尝试从历史变化估计缺失部分，这类思路叫**拟牛顿**：不用每次精确计算全部二阶导数，借助更新逐步近似。

与阻尼、信赖域结合的混合方法还需要规定何时切换、何时重置、失败后怎样恢复。不是给 $J^T J$ 随便加张表就完成了一个可靠算法。

本附录只有可分问题和割线条件两个可运行实验。原参考书的完整混合 LM、割线 LM、割线狗腿，以及大规模稀疏求解器，没有在本版实现。继续读原书 3.4—3.6 节时，先问它在近似哪张表、用什么观测更新、失败时如何恢复，再去记名称。

C.5 检查题

问题一：刚才根本没移动，即 $s = 0$，还能代入割线更新吗?

解答：不能，分母为零。非常小的步也会把噪声放大，应跳过或重建影响表。

问题二：外层参数少了，变量投影就一定更快更好吗?

解答：不一定。每次还要求内层，内层可能有约束、秩变化或昂贵计算。应按同一目标和精度比较实际结果与成本，不能只按参数个数排优劣。

附录 D　参考来源、对应关系与勘误

本教材借鉴可执行教材的组织方式，并按自学顺序重构最小二乘内容。它与保留原章节的中文译本用途不同：教材先补线性基础，再用实验解释非线性算法；译本保留原著结构，供逐节核对。

D.1　主要来源

1. Roger R. Labbe Jr., *Kalman and Bayesian Filters in Python*。项目地址：Labbe 项目 。参考其将文字、公式、代码、图形和练习解答放在 Notebook 中的教学方式。原项目正文采用 CC BY 4.0、代码采用 MIT 许可；本教材没有复制其章节正文或示例代码。
2. K. Madsen, H. B. Nielsen, O. Tingleff, *Methods for Non-Linear Least Squares Problems*, 2nd ed., April 2004。官方入口：DTU 官方入口 。
3. NumPy 官方文档：numpy.linalg.lstsq，说明最小范数解、秩、奇异值及残差摘要的返回条件：NumPy 官方说明 。
4. SciPy 官方文档：scipy.optimize.least_squares，说明 trf/lm/dogbox、边界、损失与尺度：SciPy 官方说明 。

依赖 API 随版本可能变化，复现优先使用仓库记录的验证版本，库的当前文档用于进一步查询。

D.2　只想学会使用，先读到哪里

第一次阅读可以停在主要来源这里。下面的版本对应和历史勘误，是为需要核对原著的读者保留的记录，不是学习最小二乘的前置知识。

“勘误”是指出原资料中可能的印刷或记法问题；“维度检查”是核对一张表的行列和结果数量能否对得上；“符号约定”是先说清正负、字母各代表什么。它们帮助我们避免把一个式子抄对了外形，却用错意思。

D.3　与 2004 年原书的对应

本教材	原书位置	阅读重点
01—06 章	作为新增基础；附录 B相关	线性建模、稳定解法和噪声
07 章	第 1 章与第 3 章开头	残差、梯度、Hessian、线性化

本教材	原书位置	阅读重点
08 章	第 2 章 2.1—2.3 节	下降、牛顿、线搜索、局部收敛
09 章	第 3.1 节	GN 及其适用条件
10 章	第 3.2 节	LM 阻尼与增益比
11 章	第 2.4、3.3 节	信赖域、Cauchy 与狗腿
附录 C	第 3.4—3.7 节	混合、割线和可分问题
附录 A、第 04 章	附录 A/B	正定性、SVD 和最小范数

本书统一 $r = f - y$，原书部分使用 $y - f$；引用时先检查符号约定。原书页码与中文重排页码不同，精确定位以参考工程中的检查记录和页源映射为准。

D.4 如何处理原书已记录的疑点

归档译本采取忠实保留原文并单列疑点的方式。教材采用明确推导与维度检查，不把疑似误植当作算法规范。例如：

- 原书算法 3.21 及相邻叙述中的 Cauchy 步缩放和预测增益分支存在记录在案的记法疑点。第 11 章直接从二次模型计算预测下降。
- 原书附录式 (B.3) 的外积转置有维度疑问。教材 SVD 表达式保持 $u_j^T y$ 是标量、v_j 是参数方向的维度一致性。
- 原书的图号缺口、个别下标及绝对值符号问题保留在译本的检查报告里，不在教材里重新编号成“原书修订版”。

完整历史记录保存在 references/nonlinear-least-squares/source/Full_Check_Report.md。当前单独提供的中文版 PDF 与工程自带 PDF 字节不同，归档 README 和 SHA256 清单分别标识；本次不宣称已重新逐段校对两份译本。

D.5 验证记录和适用范围

初版曾用独立 IPython 进程逐章执行，保存真实文本与图形，当时未验证 Jupyter 内核通信。这是历史记录，不是所有环境都禁止 Jupyter。2026-11-06 的云环境设置已验证 JupyterLab 读取实际 Notebook，以及真实内核执行一次数值检查；本轮改写后的逐章执行结果另记在 docs/editorial-validation.json。

本轮重点是把直觉、术语和实验说明讲清楚，不把数学推导删掉，也不把模拟结果包装成实测。对比依据与仍未完成的教学改进见 docs/teaching-comparison.md。中文 PDF 构建依赖已修复，更新正文已同步编译；复现方式见 docs/cloud-pdf-build.md，最终检查见 docs/pdf-validation.json。

测试验证了有限差分、白化方向、解与参考结果、拒绝/停止和递推等价性；它们不构成所有输入上的正确性证明。工程案例均使用明确标注的合成数据，尚未验证真实屏幕或传感器。附录 C的完整混合/割线求解器及大规模稀疏实现留作后续扩展。

D.6　检查题

问题：引用公式前，除了看字母长得一样，还要核对什么？

解答：误差是“模型减观测”还是相反，向量按行还是按列，总分有没有乘一半，单位是否一致，公式适用条件是什么，以及它前后在解哪个问题。例如某式只在没有碰到边界时成立，搬到参数被卡在零的场景就可能误导。公式旁边的解释，也是知识的一部分。